

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта

ОТЧЁТ
о лабораторной работе № 1
«Генерация бинарного кода Грея.
Мультимножества и операции над ними»
по дисциплине «Дискретная математика»

Выполнил

студент гр. 5130201/50302

Р. Ю. Дужнов

Проверил

преподаватель

А. В. Востров

«___» _____ 2026 г.

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Математическое описание	5
1.1 Множества	5
1.2 Мультимножества	5
1.3 Операции над мультимножествами	6
1.4 Универсум на коде Грея	7
1.5 Алгоритм генерации бинарного кода Грея	8
1.6 Случайный выбор подмультимножества заданной мощности .	10
2 Особенности реализации	12
2.1 Средства разработки и структура программы	12
2.2 Ограничения разрядности и вывода	13
2.3 Исключения	14
2.4 Модуль случайных чисел	15
2.5 Генерация кода Грея и носитель	15
2.6 Класс Multiset	17
2.7 Построение универсума	20
2.8 Модуль ввода	20
2.9 Класс Menu и функция main	21
3 Результаты работы	24
3.1 Сценарий 1: случайный универсум, автоматическое заполнение	24
3.2 Сценарий 2: ручной ввод с ошибками и отмена	28
3.3 Сценарий 3: универсум по мощности, граничные мощности A и B	32
3.4 Сценарий 4: некорректный ввод в меню и при создании уни- версума	35
3.5 Сценарий 5: разрядность $n = 0$	38
3.6 Сценарий 6: пересоздание универсума и конец ввода	40
3.7 Сценарий 7: наибольшая разрядность $n = 19$	43
ЗАКЛЮЧЕНИЕ	45
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	48
ПРИЛОЖЕНИЕ А Текст программы	49

ВВЕДЕНИЕ

Требуется реализовать программу, которая генерирует бинарный код Грея заданной пользователем разрядности n и заполняет им универсум мультимножеств. На основе универсума формируются два мультимножества A и B ; для каждого пользователь выбирает способ заполнения (вручную или автоматически) и задаёт мощность. Программа выводит результаты операций: объединение, пересечение, разность (оба варианта), симметрическая разность, дополнение (оба варианта), арифметические сумма, разность, произведение и деление. Требуются пользовательское меню и защита от некорректного ввода.

В работе приняты следующие уточнения постановки.

- а) Универсум U — мультимножество: каждый код Грея x входит в него с кратностью $k_U(x) \geq 1$. Кратности задаются тремя способами: случайно от 1 до K , вручную для каждого элемента или по заданной мощности $|U|$ со случайным распределением кратностей.
- б) A и B — подмультимножества U : $k_A(x) \leq k_U(x)$ для всех x . Мощность $|A|$ вводится пользователем или выбирается случайно; в обоих случаях $0 \leq |A| \leq |U|$, поэтому допустимо любое подмультимножество U , включая $\emptyset$ и сам U .
- в) Некоммутативные арифметические операции (разность и деление) вычисляются в обоих вариантах, поэтому результатов тринадцать.
- г) При $n = 0$ универсум пуст: $U = \{\}$, и все операции дают $\{\}$ (обоснование — в подразделе 1.4).

Входные данные программы перечислены в таблице 1. Пустая строка (только клавиша Enter) там, где она допустима, означает либо случайный выбор, либо отмену; в остальных местах она считается ошибкой.

Таблица 1 — Входные данные программы

Величина	Допустимые значения	Пустая строка
Пункт меню	целое от 0 до 6	ошибка
Разрядность кода Грея n	целое от 0 до 19	ошибка
Способ задания кратностей U	целое от 0 до 3 (0 — отмена)	ошибка
Наибольшая кратность K (способ 1)	целое от 1 до 100	ошибка
Кратность $k_U(x)$ (способ 2)	целое от 1 до 100	отмена создания U
Мощность $ U $ (способ 3)	целое от 2^n до $100 \cdot 2^n$	случайная мощность
Способ заполнения A, B	целое от 0 до 2 (0 — отмена)	ошибка
Мощность $ A , B $	целое от 0 до $ U $	случайная мощность
Код элемента (ручное заполнение)	строка из n символов 0 и 1	отмена заполнения
Добавляемая кратность	целое от 1 до $\min\{k_U(x) - k_A(x), A - \text{набрано}\}$	ошибка

Результат считается достигнутым, если для любых допустимых входных данных программа выводит U, A, B и тринадцать результатов операций в точном соответствии с определениями, а на любой некорректный ввод отвечает сообщением и повторным запросом, не завершаясь аварийно.

1 Математическое описание

1.1 Множества

Множество — совокупность различных элементов [1]. Если все рассматриваемые множества являются подмножествами одного множества U , его называют универсумом. Операции над $A, B \subseteq U$ [1]:

$$A \cup B = \{x \mid x \in A \text{ или } x \in B\}, \quad (1)$$

$$A \cap B = \{x \mid x \in A \text{ и } x \in B\}, \quad (2)$$

$$A \setminus B = \{x \mid x \in A \text{ и } x \notin B\} = A \cap \bar{B}, \quad (3)$$

$$A \triangle B = (A \setminus B) \cup (B \setminus A), \quad (4)$$

$$\bar{A} = U \setminus A. \quad (5)$$

Дополнение (5) определено только относительно универсума. Тожества $A \setminus B = A \cap \bar{B}$ и (4) переносятся на мультимножества и используются в реализации.

1.2 Мультимножества

Мультимножество $\hat{X}$ над множеством $X = \{x_1, \dots, x_s\}$ — совокупность элементов X , в которую x_i входит $a_i \geq 0$ раз [1]. Множество X называют носителем, число a_i — кратностью (показателем) элемента x_i . Запись:

$$\hat{X} = \{x_1^{a_1}, \dots, x_s^{a_s}\}, \quad (6)$$

элементы с нулевой кратностью не записываются. Мультимножество A над X однозначно задаётся функцией кратности $k_A: X \rightarrow \{0, 1, 2, \dots\}$, а его мощность — сумма кратностей:

$$|A| = \sum_{x \in X} k_A(x). \quad (7)$$

Мультимножество, все кратности которого равны 0 или 1, совпадает с обычным множеством.

Универсум U в данной работе тоже мультимножество над носителем X с кратностями $k_U(x) \geq 1$. Мультимножество A называется подмультимноже-

ством U ($A \subseteq U$), если

$$k_A(x) \leq k_U(x) \quad \text{для всех } x \in X. \quad (8)$$

Мультимножества A и B рассматриваются только как подмультимножества U . Из (7) и (8) следует $0 \leq |A| \leq |U|$, причём каждое значение из этого отрезка достигается: подмультимножество мощности m получается, если взять любые m из $|U|$ «единиц кратности» универсума.

1.3 Операции над мультимножествами

Все операции выполняются поэлементно: кратность x в результате зависит только от $k_A(x)$, $k_B(x)$ и $k_U(x)$ [1, 4]. Функции кратности результатов приведены в таблице 2.

Таблица 2 — Операции над мультимножествами

Операция	Обозначение	Кратность x в результате
Объединение	$A \cup B$	$\max\{k_A(x), k_B(x)\}$
Пересечение	$A \cap B$	$\min\{k_A(x), k_B(x)\}$
Дополнение	$\bar{A}$	$k_U(x) - k_A(x)$
Разность	$A \setminus B$	$\min\{k_A(x), k_U(x) - k_B(x)\}$
Симметрическая разность	$A \triangle B$	$\max\{\min\{k_A(x), k_U(x) - k_B(x)\}, \min\{k_B(x), k_U(x) - k_A(x)\}\}$
Арифметическая сумма	$A + B$	$\min\{k_A(x) + k_B(x), k_U(x)\}$
Арифметическая разность	$A - B$	$\max\{k_A(x) - k_B(x), 0\}$
Арифметическое произведение	$A * B$	$\min\{k_A(x) \cdot k_B(x), k_U(x)\}$
Арифметическое деление	A/B	$\lfloor k_A(x)/k_B(x) \rfloor$ при $k_B(x) > 0$; 0 при $k_B(x) = 0$

Формула разности получена из $A \setminus B = A \cap \bar{B}$, симметрической разности — из (4). Арифметическая разность не зависит от универсума и отличается от теоретико-множественной: например, при $k_U(x) = 5$, $k_A(x) = 3$, $k_B(x) = 1$ имеем $k_{A \setminus B}(x) = \min\{3, 4\} = 3$, но $k_{A-B}(x) = 2$. Сумма и произведение ограничены сверху кратностью $k_U(x)$, иначе результат мог бы не быть подмультимножеством U и к нему нельзя было бы применить дополнение.

Утверждение. Если $A, B \subseteq U$, то результат каждой операции из таблицы 2 является подмультимножеством U .

Действительно: $\max\{k_A(x), k_B(x)\} \leq k_U(x)$ и $\min\{k_A(x), k_B(x)\} \leq k_U(x)$ в силу (8); $0 \leq k_U(x) - k_A(x) \leq k_U(x)$; разность и симметрическая разность не больше $k_U(x) - k_B(x)$ или $k_U(x) - k_A(x)$; сумма и произведение ограничены явно; $\max\{k_A(x) - k_B(x), 0\} \leq k_A(x)$; $\lfloor k_A(x)/k_B(x) \rfloor \leq k_A(x)$ при $k_B(x) \geq 1$.

Разность, арифметическая разность и деление некоммутативны, дополнение одноместно, поэтому вычисляются оба варианта: $A \setminus B$ и $B \setminus A$, $\bar{A}$ и $\bar{B}$, $A - B$ и $B - A$, A/B и B/A . Вместе с $A \cup B$, $A \cap B$, $A \triangle B$, $A + B$ и $A * B$ это тринадцать результатов.

Для проверки результатов используются следствия определений:

$$|\bar{A}| = |U| - |A|, \quad |A \cup B| + |A \cap B| = |A| + |B|. \quad (9)$$

Второе равенство верно поэлементно: $\max\{a, b\} + \min\{a, b\} = a + b$.

1.4 Универсум на коде Грея

Носитель X — множество всех 2^n двоичных слов длины n , записанных в порядке кода Грея (подраздел 1.5); элемент универсума с номером i — i -е слово кода. Кратность каждого элемента в универсуме лежит в отрезке $[1, K_{\max}]$, $K_{\max} = 100$; поэтому

$$2^n \leq |U| \leq K_{\max} \cdot 2^n. \quad (10)$$

Кратности задаются одним из трёх способов:

- а) случайно: $k_U(x) = 1 + \xi_x$, где ξ_x независимы и равновероятны на $\{0, \dots, K-1\}$, $1 \leq K \leq K_{\max}$; тогда $k_U(x)$ равновероятна на $[1, K]$, а математическое ожидание $|U|$ равно $2^n(K+1)/2$;
- б) вручную: пользователь вводит $k_U(x) \in [1, K_{\max}]$ для каждого из 2^n элементов;
- в) по мощности: пользователь задаёт $|U|$ из (10) (или она выбирается равновероятно из этого отрезка); каждый элемент получает кратность 1, а остальные $|U| - 2^n$ единиц распределяются случайно между элементами так, чтобы ни одна кратность не превысила $K_{\max}$ (алгоритм подраздела 1.6, применённый к мультимножеству, в котором все кратности равны $K_{\max} - 1$).

При $K = 1$ (способ 1) или $|U| = 2^n$ (способ 3) все кратности равны 1, U и его подмультимножества становятся обычными множествами, а арифметиче-

ские сумма, разность и произведение совпадают с объединением, разностью и пересечением.

Случай $n = 0$. Двоичное слово длины 0 не содержит ни одного разряда. Такое слово нельзя ни показать в таблице универсума, ни ввести с клавиатуры как код элемента (пустая строка в программе означает отмену ввода). Поэтому в программе принято, что при $n = 0$ носитель пуст: $X = \emptyset$, $U = \{\}$, $|U| = 0$. Из (8) следует, что единственное подмультимножество пустого универсума — пустое, и допустимая мощность A и B равна только 0. Все функции кратности из таблицы 2 определены на пустом носителе, поэтому каждая из тринадцати операций даёт $\{\}$ с мощностью 0; соотношения (9) выполняются: $0 = 0 - 0$ и $0 + 0 = 0 + 0$. Программа проходит этот случай теми же функциями, что и общий (сценарий 5 в подразделе 3.5).

1.5 Алгоритм генерации бинарного кода Грея

Бинарный код Грея порядка n — последовательность всех 2^n двоичных слов длины n , в которой соседние слова, а также последнее и первое, различаются ровно в одном разряде [1].

Слово рассматривается как битовая шкала $B = (B[n], \dots, B[2], B[1])$; разряды нумеруются справа, $B[1]$ — младший. На шаге i инвертируется разряд с номером $Q(i)$, где

$$Q(i) = \text{число нулей в конце двоичной записи } i, \text{ увеличенное на } 1. \quad (11)$$

$Q(i)$ вычисляется так: $q := 1, j := i$; пока j чётно, $j := j/2$ и $q := q + 1$; результат — q .

Вход: целое $n \geq 1$.

Выход: последовательность из 2^n шкал длины n .

1. Положить $B[k] := 0$ для $k = 1, \dots, n$ и выдать B .
2. Для $i = 1, 2, \dots, 2^n - 1$: $p := Q(i)$; $B[p] := 1 - B[p]$; выдать B .

При $i < 2^n$ двоичная запись i оканчивается не более чем $n - 1$ нулями, поэтому $1 \leq Q(i) \leq n$ и номер разряда всегда допустим. Протокол работы при $n = 3$ — в таблице 3.

Таблица 3 — Протокол алгоритма при $n = 3$

Шаг i	Двоичная запись i	$p = Q(i)$	Шкала B
—	—	—	000
1	001	1	001
2	010	2	011
3	011	1	010
4	100	3	110
5	101	1	111
6	110	2	101
7	111	1	100

Корректность. Соседние шкалы различаются одним разрядом по построению. Докажем, что все 2^n шкал различны и последняя отличается от первой одним разрядом. Обозначим $S_k = (Q(1), \dots, Q(2^k - 1))$ — последовательность номеров инвертируемых разрядов; она не зависит от n . Для $0 < j < 2^k$ числа $2^k + j$ и j оканчиваются одинаковым числом нулей, а $Q(2^k) = k + 1$; значит,

$$S_{k+1} = (S_k, k + 1, S_k). \quad (12)$$

Индукцией по k отсюда следует, что S_k совпадает со своим обращением (палиндром): $S_1 = (1)$, и обращение $(S_k, k+1, S_k)$ равно $(S_k^R, k+1, S_k^R) = (S_k, k+1, S_k)$.

Пусть при длине k алгоритм выдаёт шкалы $g_1, \dots, g_m$, $m = 2^k$, и утверждение для них верно. При длине $k + 1$ первые $m - 1$ шагов используют S_k и не трогают старший разряд $k + 1$: получаются $0g_1, \dots, 0g_m$. Шаг m инвертирует старший разряд: $1g_m$. Оставшиеся $m - 1$ шагов снова применяют S_k , то есть, так как S_k — палиндром, обращение S_k , и проходят шкалы в обратном порядке: $1g_{m-1}, \dots, 1g_1$. Итог — $(0g_1, \dots, 0g_m, 1g_m, \dots, 1g_1)$, отражённый код Грея. Шкалы внутри каждой половины различны по предположению индукции, половины различаются старшим разрядом, последняя шкала $1g_1$ отличается от первой $0g_1$ только старшим разрядом. База $k = 1$: шкалы 0 и 1. Получаемая последовательность совпадает с замкнутой формой $g(i) = i \oplus \lfloor i/2 \rfloor$;

программа проверялась на совпадение при $n = 1, \dots, 16$.

Трудоёмкость. Алгоритм делает $2^n - 1$ шагов; на шаге i вычисление $Q(i)$ требует $Q(i)$ проверок чётности. Так как

$$\sum_{i=1}^{2^n-1} Q(i) = (2^n - 1) + \sum_{j=1}^n \left\lfloor \frac{2^n - 1}{2^j} \right\rfloor = 2^{n+1} - n - 2,$$

общее время — $O(2^n)$, то есть $O(1)$ в среднем на слово. Память — $O(2^n)$ машинных слов при хранении каждой шкалы целым числом.

1.6 Случайный выбор подмультимножества заданной мощности

Автоматическое заполнение A мощности m и способ 3 задания универсума сводятся к одной задаче: дано мультимножество-ограничение V (это U или «свободные места» универсума); выбрать случайное $A \subseteq V$ с $|A| = m$, $0 \leq m \leq |V|$.

Используется урновая модель. Элемент x представлен $k_V(x)$ одинаковыми шарами с меткой x , всего шаров $N = |V|$. Из урны без возвращения извлекаются m шаров, и $k_A(x)$ — число извлечённых шаров с меткой x . Выбор m шаров из N выполняется алгоритмом выборки S [2]: шары просматриваются по очереди, и текущий шар берётся с вероятностью

$$P = \frac{m - s}{N - t}, \quad (13)$$

где t — число уже просмотренных шаров, s — число уже взятых.

Вход: V с кратностями $k_V(x)$, $N = |V|$; целое m .

Выход: $A \subseteq V$, $|A| = m$.

1. Если $m > N$, завершить с ошибкой.
2. Положить $k_A(x) := 0$ для всех x , $need := m$, $left := N$.
3. Для элементов x по порядку, пока $need > 0$: повторить $k_V(x)$ раз, пока $need > 0$: если $need = left$ или случайное целое $r \in [0, left - 1]$ меньше $need$, то $k_A(x) := k_A(x) + 1$ и $need := need - 1$; затем $left := left - 1$.
4. Вернуть A .

Условие $r < need$ при равновероятном r выполняется с вероятностью $need/left$, что совпадает с (13).

Корректность. Всегда $0 \leq need \leq left$: при $need = left$ шар берётся обязательно. Поэтому к концу просмотра взято ровно m шаров, $|A| = m$, а из каждого элемента взято не больше $k_V(x)$ шаров, то есть $A \subseteq V$. Вероятность выбрать заданный набор из m шаров равна произведению вероятностей (13) по всем просмотренным шарам; числители выбранных шаров дают $m!$, числители невыбранных — $(N - m)!$, знаменатели — $N!$:

$$P(\text{набор}) = \frac{m! (N - m)!}{N!} = \binom{N}{m}^{-1}, \quad (14)$$

то есть все $\binom{N}{m}$ наборов шаров равновероятны. Следовательно, $k_A(x)$ имеет гипергеометрическое распределение, и

$$M k_A(x) = m \cdot \frac{k_V(x)}{N} : \quad (15)$$

элементы с большей кратностью в универсуме получают в среднем пропорционально большую кратность в A .

Трудоёмкость. Просматривается не больше $N = |V|$ шаров, на каждый — одно случайное число; после того как $need$ обратится в 0 или сравняется с $left$, случайные числа не нужны. Время — $O(|V| + 2^n)$, без холостых выборов и с гарантированной верхней границей. При $|V| \leq K_{\max} \cdot 2^n$ это $O(K_{\max} \cdot 2^n)$; именно эта величина определяет наибольшую допустимую разрядность (подраздел 2.2).

2 Особенности реализации

2.1 Средства разработки и структура программы

Язык C++14, только стандартная библиотека; компилятор GCC 6.3.0 (MinGW, 32-разрядная сборка), ОС Windows 11. Сборка:

```
g++ -std=c++14 -O2 -Isrc src/main.cpp src/ui/*.cpp  
src/core/*.cpp -o lab1
```

или через CMake (файл CMakeLists.txt, цели lab1 и bench). Предупреждений при -Wall -Wextra нет.

Код разделён на два слоя: core — математическая часть, не работающая с консолью; ui — ввод и меню. Зависимости направлены только от ui к core. Файлы перечислены в таблице 4, текст программы — в приложении А.

Таблица 4 — Файлы программы

Файл	Назначение
src/core/GrayCode.hpp, .cpp	генерация кода Грея, класс CodeSet (носитель)
src/core/Multiset.hpp, .cpp	класс Multiset, операции, случайный выбор подмультимножества, вывод
src/core/Universe.hpp, .cpp	три способа построения универсума
src/core/Random.hpp, .cpp	генератор случайных чисел
src/ui/Input.hpp, .cpp	защищённый построчный ввод
src/ui/Menu.hpp, .cpp	класс Menu: состояние программы и меню
src/main.cpp	точка входа, ключ --seed
tools/bench.cpp	замер времени для выбора наибольшей разрядности

Все величины задачи (разрядность, номера, кратности, мощности, коды) неотрицательны и хранятся в типе `size_t`. В использованной 32-разрядной сборке `size_t` занимает 4 байта (наибольшее значение $2^{32} - 1 = 4\,294\,967\,295$). Наибольшая мощность универсума $100 \cdot 2^{19} = 52\,428\,800$, наибольшее произведение кратностей $100 \cdot 100$, наибольшее вводимое число 999 999 999 — переполнение невозможно. Вычитание беззнаковых величин выполняется только тогда, когда уменьшаемое заведомо не меньше: в дополнении это гарантирует проверка $A \subseteq U$, в арифметической разности — сравнение $k_A(x) > k_B(x)$, в ручном заполнении — инвариант $k_A(x) \leq k_U(x)$.

2.2 Ограничения разрядности и вывода

Наибольшая разрядность $n_{\max} = 19$ (константа `MAX_DEPTH`). Критерий выбора: любое действие пользователя должно завершаться быстрее 1 с. Это граница времени отклика, при которой пользователь ещё воспринимает работу с программой как непрерывную [3]; при большей задержке интерактивное меню «зависает» в восприятии пользователя.

Самое долгое действие — случайный выбор подмультимножества (подраздел 1.6): его время пропорционально $|U| \leq K_{\max} \cdot 2^n$. Остальные действия линейны по 2^n с малой константой. Время измерено утилитой `tools/bench.cpp` в наихудшем случае: все $k_U(x) = K_{\max} = 100$, мощность $|A| = |U|/2$ (при ней алгоритм просматривает почти все $|U|$ шаров), лучший из трёх запусков. Машина: Intel Core i7-13700KF, 32 ГБ ОЗУ; сборка -O2. Результаты — в таблице 5.

Таблица 5 — Время действий, мс (наихудший случай, $K_{\max} = 100$)

n	Код Грея	U случайно (способ 1)	U по мощности (способ 3)	Заполнение A	13 операций
16	0,0	0,0	72	72	2,0
17	0,0	0,6	144	145	3,6
18	0,5	1,5	288	381	12
19	1,0	6,0	873	877	23
20	3,0	13	1752	1746	49
21	7,0	25	3551	3529	125

Время растёт примерно вдвое на каждый разряд, так как вдвое растёт $|U|$. При $n = 19$ наибольшее время 0,88 с укладывается в 1 с, при $n = 20$ оно уже 1,75 с. Поэтому $n_{\max} = 19$. Память не ограничивает: при $n = 19$ один объект `Multiset` занимает $2^{19} \cdot 4$ байт = 2 МиБ, носитель — 4 МиБ.

Независимо от критерия времени есть предел типа: 2^n должно помещаться в `size_t`, поэтому `generate_gray_code` отвергает $n > 31$ (константа `MAX_TYPE_DEPTH`). `MAX_DEPTH` — ограничение интерфейса, `MAX_TYPE_DEPTH` — ограничение ядра; утилита замера пользуется ядром и потому может измерять $n > 19$.

Наибольшая кратность $K_{\max} = 100$ (константа

MAX_MULTPLICITY): трёхзначные кратности умецаются в столбцы таблиц. От $K_{\max}$ зависит $n_{\max}$: время автозаполнения пропорционально $K_{\max} \cdot 2^n$, поэтому при изменении $K_{\max}$ замер нужно повторить.

Ограничения вывода. Таблица выводит не больше 32 строк (TABLE_LIMIT; при $n \leq 5$ — полностью), запись мультимножества — не больше 16 элементов (PRINT_LIMIT; при $n \leq 4$ — всегда полностью). Дальше выводится пометка, сколько строк или элементов не показано, а мощности выводятся всегда полностью. При $n = 19$ полный вывод одного результата занял бы около 2^{19} записей и был бы необозрим.

2.3 Исключения

Ошибки пользователя исключений не вызывают: программа выводит сообщение и повторяет запрос. Исключения сигнализируют о нарушении внутренних ограничений, которое возможно только при ошибке в самой программе, и о конце потока ввода (таблица 6).

Таблица 6 — Классы исключений

Класс	Родитель	Условие
GrayCodeError	std::runtime_error	общий родитель ошибок кода Грея
InvalidDepthError	GrayCodeError	$n > \text{MAX_TYPE_DEPTH}$
CodeIndexError	GrayCodeError	номер кода вне носителя
MultisetError	std::runtime_error	общий родитель ошибок мультимножеств
MultisetIndexError	MultisetError	номер элемента вне носителя
CarrierMismatchError	MultisetError	операнды построены над разными носителями
NotSubmultisetError	MultisetError	операнд не является подмультимножеством U
CardinalityOverflowError	MultisetError	требуемая мощность больше мощности ограничения
InvalidMultiplicityError	MultisetError	$K \notin [1, K_{\max}]$
UniverseCardinalityError	MultisetError	$ U $ вне (10)
InputClosedError	std::runtime_error	поток ввода закончился

2.4 Модуль случайных чисел

Источник случайности — генератор `std::mt19937` (вихрь Мерсенна, 32-разрядные значения); равномерность обеспечивает `std::uniform_int_distribution` [5]. Стандартная `std::rand` не подходит: в использованной сборке `RAND_MAX = 32767`, и при $|U| > 32768$ часть шаров в алгоритме подраздела 1.6 была бы недостижима.

- `unsigned make_seed()` — возвращает зерно: `std::random_device` *исключающее ИЛИ* текущее время. В MinGW `std::random_device` детерминирован (проверено: два запуска выдают одинаковые числа), поэтому без времени все запуски давали бы одни и те же универсумы.
- `void seed_random(unsigned seed)` — задаёт начальное состояние генератора.
- `size_t random_below(size_t bound)` — вход: `bound > 0`; выход: равновероятное целое из $[0, bound - 1]$.

Функция `main` выводит использованное зерно; запуск `lab1 --seed N` повторяет все случайные числа. Так получены воспроизводимые сценарии раздела 3.

2.5 Генерация кода Грея и носитель

Шкала B хранится одним целым числом: разряд $B[p]$ — бит числа с номером $p - 1$ (нумерация справа с нуля), инвертирование — `code ^= 1 << (p - 1)`. Реализация — в листинге 1.

Листинг 1 — Генерация кода Грея (`GrayCode.cpp`)

```
4 // функция Q из алгоритма: номер разряда (справа, с единицы), который
   меняется на шаге i,
5 // то есть число нулей в конце двоичной записи i плюс один
6 static size_t changed_bit(size_t i){
7     size_t q = 1;
8     size_t j = i;
9     while(j % 2 == 0){
10         j /= 2;
11         q++;
12     }
13     return q;
14 }
15
```

```

16 std::vector<size_t> generate_gray_code(size_t depth){
17     if(depth > MAX_TYPE_DEPTH){
18         throw InvalidDepthError();
19     }
20     std::vector<size_t> codes;
21     // слово длины 0 не содержит разрядов: в программе принято, что при n = 0
    кодов нет
22     if(depth == 0){
23         return codes;
24     }
25     size_t count = size_t(1) << depth;
26     codes.reserve(count);
27     size_t code = 0;                                // шкала B = 00...0
28     codes.push_back(code);
29     for(size_t i = 1; i < count; i++){
30         size_t p = changed_bit(i);                    // номер разряда справа, 1 <=
        p <= depth
31         code ^= size_t(1) << (p - 1);                // B[p] := 1 - B[p]
32         codes.push_back(code);
33     }
34     return codes;
35 }

```

- static size_t changed_bit(size_t i) — функция $Q(11)$; вход: $i > 0$; выход: $Q(i)$. Видна только в GrayCode.cpp.
- std::vector<size_t> generate_gray_code(size_t depth) — вход: n ; выход: 2^n кодов в порядке построения; при $n = 0$ — пустая последовательность; при $n > 31$ — исключение InvalidDepthError.
- std::string code_to_string(size_t code, size_t depth) — выход: строка из n символов 0 и 1, старший разряд слева.

Класс CodeSet — носитель мультимножеств (таблицы 7, 8). Поле positions — обратная таблица: positions[c] — номер кода c в последовательности. Так как код Грея содержит все 2^n слов, номер есть у каждого слова длины n , и поиск элемента по введённой строке выполняется за $O(n)$ (перевод строки в число и одно обращение к массиву) вместо $O(n \cdot 2^n)$ при просмотре всех кодов.

Таблица 7 — Поля класса CodeSet

Поле	Тип	Назначение
depth	size_t	разрядность n
codes	std::vector<size_t>	codes[i] — i -й код Грея
positions	std::vector<size_t>	positions[c] — номер кода c

Таблица 8 — Методы класса CodeSet

Метод	Вход	Выход
CodeSet(depth)	разрядность n	носитель из 2^n кодов (пустой при $n = 0$)
get_depth()	—	n
size()	—	число элементов 2^n (0 при $n = 0$)
get_code(i)	номер i	строка кода; CodeIndexError при $i \geq 2^n$
find(code, index)	строка; переменная для номера	true и номер в index, если строка — слово длины n из 0 и 1; иначе false

2.6 Класс Multiset

Класс Multiset — мультимножество над носителем, представленное функцией кратности. Объектами этого класса являются и универсум U , и мультимножества A , B , и все результаты операций: универсум отличается только тем, что его кратности не меньше 1 и он передаётся операциям, которым нужна $k_U(x)$. Поля — в таблице 9.

Таблица 9 — Поля класса Multiset

Поле	Тип	Назначение
carrier	std::shared_ptr<const CodeSet>	общий носитель; объект носителя живёт, пока на него ссылается хотя бы одно мультимножество
counts	std::vector<size_t>	counts[i] — кратность i -го кода Грея

Кратности хранятся по номерам элементов носителя, поэтому поиск элементов в операциях не нужен, и каждая операция выполняется за $O(2^n)$ одним проходом (разность — двумя, симметрическая разность — пятью, так как они собраны из других операций). Совместное владение носителем через `shared_ptr` исключает висячие указатели: при пересоздании универсума старые мультимножества продолжают ссылаться на свой носитель. Принадлежность одному носителю проверяется сравнением указателей.

Методы доступа — в таблице 10. Закрытый метод `check_same_carrier` вызывается каждой двуместной операцией, `check_within` — каждой операцией, которой передаётся универсум.

Таблица 10 — Методы доступа класса `Multiset`

Метод	Вход	Выход
<code>Multiset(carrier)</code>	носитель	мультимножество с нулевыми кратностями
<code>size()</code>	—	число элементов носителя
<code>get_count(i)</code>	номер	кратность; <code>MultisetIndexError</code>
<code>set_count(i, count)</code>	номер, кратность	кратность изменена; <code>MultisetIndexError</code>
<code>cardinality()</code>	—	мощность по (7)
<code>is_submultiset_of(other)</code>	мультимножество	истина, если выполнено (8)
<code>check_same_carrier(other)</code>	мультимножество	<code>CarrierMismatchError</code> , если носители разные
<code>check_within(universe)</code>	универсум	то же, плюс <code>NotSubmultisetError</code> , если объект не подмультимножество U

Методы операций (таблица 11): первый операнд A — сам объект, второй B — параметр; каждый метод возвращает новый объект и не меняет операнды. Метод объединения назван `unite`, так как `union` — ключевое слово C++.

Таблица 11 — Методы операций класса Multiset

Метод	Результат	Реализация
<code>unite(b)</code>	$A \cup B$	max по элементам
<code>intersect(b)</code>	$A \cap B$	min по элементам
<code>complement(u)</code>	$\bar{A}$	$k_U(x) - k_A(x)$ после проверки $A \subseteq U$
<code>difference(b, u)</code>	$A \setminus B$	$A \cap \bar{B}$ методами <code>intersect</code> и <code>complement</code>
<code>symmetric_difference(b, u)</code>	$A \triangle B$	$(A \setminus B) \cup (B \setminus A)$ методами <code>difference</code> и <code>unite</code>
<code>arithmetic_sum(b, u)</code>	$A + B$	$\min\{k_A(x) + k_B(x), k_U(x)\}$
<code>arithmetic_difference(b)</code>	$A - B$	$k_A(x) - k_B(x)$ при $k_A(x) > k_B(x)$, иначе 0
<code>arithmetic_product(b, u)</code>	$A * B$	$\min\{k_A(x) \cdot k_B(x), k_U(x)\}$
<code>arithmetic_division(b)</code>	A/B	$\lfloor k_A(x)/k_B(x) \rfloor$ при $k_B(x) \neq 0$, иначе 0

Функция `random_submultiset(bound, cardinality)` реализует алгоритм подраздела 1.6 (листинг 2); вход: ограничение V и мощность m ; выход: случайное $A \subseteq V$, $|A| = m$; при $m > |V|$ — `CardinalityOverflowError`. Она объявлена другом класса, чтобы работать с `counts` напрямую.

Листинг 2 — Случайный выбор подмультимножества (`Multiset.cpp`)

```

135 Multiset random_submultiset(const Multiset& bound, size_t cardinality){
136     size_t left = bound.cardinality();    // единиц, которые ещё не
        просмотрены
137     if(cardinality > left){
138         throw CardinalityOverflowError();
139     }
140     // элемент x_i — это bound[i] одинаковых шаров в урне; единицы
        просматриваются по очереди,
141     // и каждая берётся с вероятностью need / left (алгоритм S, Кнут, т. 2,
        п. 3.4.2)
142     Multiset res(bound.carrier);
143     size_t need = cardinality;
144     for(size_t i = 0; i < bound.counts.size() && need > 0; i++){
145         size_t taken = 0;
146         for(size_t unit = 0; unit < bound.counts[i] && need > 0; unit++){
147             if(need == left || random_below(left) < need){
148                 taken++;
149                 need--;
150             }
        }
    }

```

```

151         left--;
152     }
153     res.counts[i] = taken;
154 }
155 return res;
156 }

```

Вывод: `print_multiset(out, multiset, limit)` записывает в поток элементы с ненулевой кратностью в виде `{000(2), 011(1)}`, не больше `limit` элементов, и пометку «... ещё N эл.»; пустое мультимножество записывается как `{}`. Оператор `<<` выводит мультимножество полностью.

Деструкторы, конструкторы копирования и перемещения не объявлены явно: классы содержат только `std::vector`, `std::shared_ptr` и числа, и сгенерированные компилятором функции корректны.

2.7 Построение универсума

Модуль `Universe` реализует способы 1 и 3 подраздела 1.4; способ 2 (ручной) выполняет меню, записывая введённые числа методом `set_count`.

- `min_universe_cardinality(size)` и
`max_universe_cardinality(size)` — границы (10): 2^n и $K_{\max} \cdot 2^n$.
- `random_universe(carrier, K)` — вход: носитель, $K \in [1, K_{\max}]$; выход: универсум с $k_U(x) = 1 + \text{random_below}(K)$; иначе `InvalidMultiplicityError`.
- `universe_with_cardinality(carrier, m)` — вход: носитель, m из (10); выход: универсум мощности m : строится мультимножество `extra` с кратностями $K_{\max} - 1$, из него выбирается `random_submultiset` мощности $m - 2^n$, и к каждой кратности прибавляется 1; иначе `UniverseCardinalityError`.

2.8 Модуль ввода

Весь ввод читается построчно функцией `std::getline [6]` и проверяется целиком. Оператор `>>` не подходит: он принимает «12abc» как 12, превращает «-1» в большое беззнаковое число и после ошибки оставляет поток в состоянии сбоя, из-за чего программа за циклируется.

- `static std::string read_line(prompt)` — печатает приглашение, читает строку, убирает пробелы, табуляции и `\r` по краям; при конце потока — `InputClosedError`. Если ввод перенаправлен из файла (стандартный ввод не консоль), функция выводит прочитанную строку, чтобы протокол выглядел как сеанс в консоли.
- `static bool parse_number(text, value)` — истина, если строка состоит из 1–9 цифр; число записывается в `value`. Отвергаются знаки, точки, пробелы внутри и пустая строка.
- `static bool is_code(text, depth)` — истина, если строка длины n из символов 0 и 1.
- `size_t read_number(prompt, min, max)` — повторяет запрос с сообщением «Ошибка: введите целое число от `min` до `max`», пока не получит число из `[min, max]`.
- `bool read_number_or_empty(prompt, min, max, value)` — то же, но на пустую строку возвращает `false`.
- `bool read_code_or_empty(prompt, depth, code)` — повторяет запрос с сообщением «Ошибка: код должен состоять из n символов 0 и 1», пока не получит код; на пустую строку возвращает `false`.

2.9 Класс Menu и функция main

Класс `Menu` хранит состояние программы: общий носитель `carrier`, мультимножества `universe`, `a`, `b` и признаки `universe_ready`, `a_ready`, `b_ready`. Признак создания универсума нужен отдельно от его пустоты: при $n = 0$ универсум создан, но пуст.

Меню выводит состояние (n , число элементов, $|U|$, $|A|$, $|B|$ или «не заполнено») и пункты: 1 — создать U ; 2, 3 — заполнить A , B ; 4 — таблица кратностей U , A , B ; 5 — операции; 6 — таблица кратностей всех тринадцати результатов; 0 — выход. Пункты 2–4 требуют созданного универсума, пункты 5 и 6 — ещё и заполненных A и B ; иначе выводится подсказка «Сначала создайте универсум (пункт 1).» или «Сначала заполните A и B (пункты 2 и 3).».

Создание универсума (`create_universe`): читается $n \in [0, 19]$ и строится новый носитель. При $n = 0$ выводится «При $n = 0$ код Грея не содержит ни одного слова: $U = \{\}$, $|U| = 0$.», и способ задания кратностей не запрашивается, так как задавать нечего. Иначе читается способ (0 — отмена, 1–3) и

его параметры. Новые объекты собираются в локальных переменных и присваиваются полям только после успешного ввода, поэтому отмена на любом шаге сохраняет прежнее состояние. После создания A и B становятся пустыми мультимножествами над новым носителем со снятыми признаками; если они были заполнены, выводится сообщение, что они построены над прежним универсумом и очищены.

Заполнение (`fill_multiset`): читается способ (0 — отмена, 1 — ручную, 2 — автоматически), затем мощность $m \in [0, |U|]$; на пустую строку m выбирается равномерно из $[0, |U|]$ и выводится. Заполняется новый объект; поле и признак обновляются только при успехе. Автоматическое заполнение — `random_submultiset(universe, m)`. Ручное заполнение (`fill_manual`):

1. Вывести таблицу универсума.
2. Пока $|A| < m$: вывести остаток $d = m - |A|$; прочитать код x (пустая строка — отмена всего заполнения).
3. Найти номер x методом `find` и вычислить свободное место $f = k_U(x) - k_A(x)$. Если $f = 0$, вывести «Ошибка: элемент x уже входит в A с наибольшей кратностью $k_U(x) = \dots$ » и вернуться к шагу 2.
4. Прочитать добавляемую кратность $c \in [1, \min\{f, d\}]$, увеличить $k_A(x)$ на c и вернуться к шагу 2.

Ограничение $c \leq \min\{f, d\}$ сохраняет $A \subseteq U$ и не даёт превысить заданную мощность; так как $m \leq |U|$, свободное место есть всегда, пока $|A| < m$. При $m = 0$ (в частности, при $n = 0$) цикл не выполняется ни разу и $A = \{\}$.

Вывод. `show_table` печатает столбцы «№», «Код», « k_U » (и « k_A », « k_B » с прочерком для незаполненного) и строку мощностей. `compute_operations` вычисляет тринадцать результатов в виде списка «название, обозначение, мультимножество»; по нему `show_operations` печатает строки «название обозначение = мультимножество, мощность m », а `show_operations_table` — таблицу кратностей всех результатов. Ширины столбцов считаются в символах, а не байтах: строки в UTF-8, и такие символы, как « $\cup$ », занимают несколько байт.

Функция `main` в Windows вызывает `SetConsoleOutputCP(CP_UTF8)`, так как консоль по умолчанию выводит в кодировке 866; задаёт зерно (`--seed N` или `make_seed`) и печатает его; запускает меню.

`InputClosedError` обрабатывается как штатное завершение: «Ввод завершён.», код возврата 0. Любое другое исключение (возможно только при ошибке в программе) — сообщение и код возврата 1.

3 Результаты работы

Работа проверена на семи сценариях. Ввод каждого сценария записан в файл `report/scenarios/*.txt`, и программа запускалась командой `lab1 --seed N < файл`, где N — номер сценария; при повторном запуске вывод совпадает до символа. Так как ввод берётся из файла, введённые строки печатает сама программа (подраздел 2.8), и протокол совпадает с тем, что видно в консоли при ручном вводе. Рисунки получены из этих протоколов скриптом `report/make_figures.py`: каждый рисунок начинается со строки выбора пункта меню; если рисунок охватывает несколько действий, меню между ними сохранено.

Кроме сценариев, ядро проверено отдельной программой: для $n = 0, \dots, 10$ и 200 случайных пар A, B на каждое n все тринадцать результатов совпали с формулами таблицы 2 поэлементно и оказались подмультимножествами U ; для $n = 1, \dots, 16$ код Грея проверен на цикличность, различие соседних слов в одном разряде и совпадение с $i \oplus \lfloor i/2 \rfloor$; доля выбора элемента x при $U = \{x^2, y^1\}$, $m = 1$ составила 0,6667 на 300 000 испытаний при теоретической $2/3$.

3.1 Сценарий 1: случайный универсум, автоматическое заполнение

После запуска программа печатает зерно и меню, в котором универсум не создан, а A и B не заполнены (рисунок 1).

```
Зерно генератора: 1 (повторить запуск: lab1 --seed 1)

=====
      Мультимножества на бинарном коде Грея
=====
U: не создан
A: не заполнено
B: не заполнено
-----
1. Создать универсум U
2. Заполнить A
3. Заполнить B
4. Таблица кратностей U, A, B
5. Операции над A и B
6. Таблица кратностей результатов операций
0. Выход
-----
Выберите пункт (0..6): 1
```

Рисунок 1 — Запуск программы, зерно 1

Создаётся универсум с $n = 3$, способ 1, $K = 5$ (рисунок 2). Коды идут в порядке таблицы 3, кратности случайны в $[1, 5]$, $|U| = 26$.

```

Выберите пункт (0..6): 1

--- Создание универсума ---
Разрядность кода Грея n (0..19): 3
Код Грея построен, число элементов 2^n = 8.
Способ задания кратностей k_U(x):
  1 - случайно, от 1 до K
  2 - вручную, для каждого элемента
  3 - по мощности |U|, кратности распределяются случайно
  0 - отмена
Способ (0..3): 1
Наибольшая кратность K (1..100): 5
Универсум создан.
  №  Код  k_U
  0   000   3
  1   001   5
  2   011   4
  3   010   5
  4   110   1
  5   111   1
  6   101   2
  7   100   5
Мощности: |U| = 26

```

Рисунок 2 — Создание универсума: $n = 3$, $K = 5$

A заполняется автоматически с заданной мощностью 10; для B мощность не вводится (Enter) и выбирается случайно: $|B| = 12$ (рисунок 3). Таблица пункта 4 (рисунок 4) подтверждает $k_A(x) \leq k_U(x)$ и $k_B(x) \leq k_U(x)$ для всех восьми элементов.

```

Выберите пункт (0..6): 2

--- Заполнение A ---
Способ заполнения:
  1 - вручную
  2 - автоматически
  0 - отмена
Способ (0..2): 2
Мощность |A| (0..26; Enter - случайная): 10
A = {000(3), 001(1), 010(3), 101(2), 100(1)}, |A| = 10

=====
      Мультимножества на бинарном коде Грея
=====
U: n = 3, элементов 8, |U| = 26
A: |A| = 10
B: не заполнено

-----
1. Создать универсум U
2. Заполнить A
3. Заполнить B
4. Таблица кратностей U, A, B
5. Операции над A и B
6. Таблица кратностей результатов операций
0. Выход
-----

Выберите пункт (0..6): 3

--- Заполнение B ---
Способ заполнения:
  1 - вручную
  2 - автоматически
  0 - отмена
Способ (0..2): 2
Мощность |B| (0..26; Enter - случайная):
Мощность выбрана случайно: |B| = 12
B = {000(2), 001(1), 011(2), 010(1), 111(1), 101(1), 100(4)}, |B| = 12

```

Рисунок 3 — Автоматическое заполнение: $|A|$ задана, $|B|$ выбрана случайно

Выберите пункт (0..6): 4

№	Код	k_U	k_A	k_B
0	000	3	3	2
1	001	5	1	1
2	011	4	0	2
3	010	5	3	1
4	110	1	0	0
5	111	1	0	1
6	101	2	2	1
7	100	5	1	4

Мощности: $|U| = 26$, $|A| = 10$, $|B| = 12$

Рисунок 4 — Таблица кратностей U, A, B

Результаты операций — на рисунке 5, таблица кратностей — на рисунке 6.

Выберите пункт (0..6): 5

--- Операции над A и B ---

$U = \{000(3), 001(5), 011(4), 010(5), 110(1), 111(1), 101(2), 100(5)\}$, $|U| = 26$
 $A = \{000(3), 001(1), 010(3), 101(2), 100(1)\}$, $|A| = 10$
 $B = \{000(2), 001(1), 011(2), 010(1), 111(1), 101(1), 100(4)\}$, $|B| = 12$

Теоретико-множественные операции:

Объединение $A \cup B = \{000(3), 001(1), 011(2), 010(3), 111(1), 101(2), 100(4)\}$, мощность 16
Пересечение $A \cap B = \{000(2), 001(1), 010(1), 101(1), 100(1)\}$, мощность 6
Разность $A \setminus B = \{000(1), 001(1), 010(3), 101(1), 100(1)\}$, мощность 7
Разность $B \setminus A = \{001(1), 011(2), 010(1), 111(1), 100(4)\}$, мощность 9
Симметрическая разность $A \Delta B = \{000(1), 001(1), 011(2), 010(3), 111(1), 101(1), 100(4)\}$, мощность 13
Дополнение $\neg A = \{001(4), 011(4), 010(2), 110(1), 111(1), 100(4)\}$, мощность 16
Дополнение $\neg B = \{000(1), 001(4), 011(2), 010(4), 110(1), 101(1), 100(1)\}$, мощность 14

Арифметические операции:

Арифметическая сумма $A + B = \{000(3), 001(2), 011(2), 010(4), 111(1), 101(2), 100(5)\}$, мощность 19
Арифметическая разность $A - B = \{000(1), 010(2), 101(1)\}$, мощность 4
Арифметическая разность $B - A = \{011(2), 111(1), 100(3)\}$, мощность 6
Арифметическое произведение $A * B = \{000(3), 001(1), 010(3), 101(2), 100(4)\}$, мощность 13
Арифметическое деление $A / B = \{000(1), 001(1), 010(3), 101(2)\}$, мощность 7
Арифметическое деление $B / A = \{001(1), 100(4)\}$, мощность 5

Рисунок 5 — Результаты тринадцати операций (сценарий 1)

Выберите пункт (0..6): 6

--- Кратности элементов в результатах операций ---

Код	U	A	B	$A \cup B$	$A \cap B$	$A \setminus B$	$B \setminus A$	$A \Delta B$	$\neg A$	$\neg B$	$A+B$	$A-B$	$B-A$	$A*B$	A/B	B/A
000	3	3	2	3	2	1	0	1	0	1	3	1	0	3	1	0
001	5	1	1	1	1	1	1	1	4	4	2	0	0	1	1	1
011	4	0	2	2	0	0	2	2	4	2	2	0	2	0	0	0
010	5	3	1	3	1	3	1	3	2	4	4	2	0	3	3	0
110	1	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0
111	1	0	1	1	0	0	1	1	1	0	1	0	1	0	0	0
101	2	2	1	2	1	1	0	1	0	1	2	1	0	2	2	0
100	5	1	4	4	1	1	4	4	4	1	5	0	3	4	0	4
$ \cdot $	26	10	12	16	6	7	9	13	16	14	19	4	6	13	7	5

Рисунок 6 — Таблица кратностей результатов (сценарий 1)

Ручной расчёт для трёх элементов по таблице 2 приведён в таблице 12 и совпадает со строками 000, 010 и 100 рисунка 6.

Таблица 12 — Ручная проверка (сценарий 1)

	000: $k_U=3, k_A=3, k_B=2$	010: $k_U=5, k_A=3, k_B=1$	100: $k_U=5, k_A=1, k_B=4$
$A \cup B, A \cap B$	3, 2	3, 1	4, 1
$A \setminus B$	$\min\{3, 1\} = 1$	$\min\{3, 4\} = 3$	$\min\{1, 1\} = 1$
$B \setminus A$	$\min\{2, 0\} = 0$	$\min\{1, 2\} = 1$	$\min\{4, 4\} = 4$
$A \triangle B$	$\max\{1, 0\} = 1$	$\max\{3, 1\} = 3$	$\max\{1, 4\} = 4$
$\overline{A}, \overline{B}$	0, 1	2, 4	4, 1
$A + B$	$\min\{5, 3\} = 3$	$\min\{4, 5\} = 4$	$\min\{5, 5\} = 5$
$A - B, B - A$	1, 0	2, 0	0, 3
$A * B$	$\min\{6, 3\} = 3$	$\min\{3, 5\} = 3$	$\min\{4, 5\} = 4$
$A/B, B/A$	$\lfloor 3/2 \rfloor = 1, \lfloor 2/3 \rfloor = 0$	3, 0	0, 4

Элемент 000 показывает ограничение универсумом: без него $k_{A+B} = 5$ и $k_{A*B} = 6$, программа выводит 3. Элемент 010 показывает различие разностей: $k_{A \setminus B} = 3, k_{A-B} = 2$. Соотношения (9): $|\overline{A}| = 26 - 10 = 16, |\overline{B}| = 26 - 12 = 14, |A \cup B| + |A \cap B| = 16 + 6 = 22 = 10 + 12$.

3.2 Сценарий 2: ручной ввод с ошибками и отмена

Универсум с $n = 2$ задаётся вручную (способ 2, рисунок 7). Для элемента 00 вводятся 0, 101 и «abc» — все отвергаются с указанием диапазона $[1, 100]$; затем вводятся кратности 2, 1, 3, 1, и $U = \{00^2, 01^1, 11^3, 10^1\}, |U| = 7$.

```

Выберите пункт (0..6): 1

--- Создание универсума ---
Разрядность кода Грея n (0..19): 2
Код Грея построен, число элементов 2^n = 4.
Способ задания кратностей k_U(x):
  1 - случайно, от 1 до K
  2 - вручную, для каждого элемента
  3 - по мощности |U|, кратности распределяются случайно
  0 - отмена
Способ (0..3): 2
Введите кратность каждого элемента (1..100); пустая строка - отмена.
k_U(00) = 0
Ошибка: введите целое число от 1 до 100
k_U(00) = 101
Ошибка: введите целое число от 1 до 100
k_U(00) = abc
Ошибка: введите целое число от 1 до 100
k_U(00) = 2
k_U(01) = 1
k_U(11) = 3
k_U(10) = 1
Универсум создан.
      №   Код   k_U
      0   00    2
      1   01    1
      2   11    3
      3   10    1
Мощности: |U| = 7

```

Рисунок 7 — Ручное задание кратностей универсума с ошибками ввода

A заполняется вручную с мощностью 4 (рисунок 8). Код «0» неверной длины и код «0a» с недопустимым символом отвергаются. Для 00 свободно $f = 2$, остаток $d = 4$, поэтому запрашивается кратность из $[1, 2]$; ввод 3 отвергается. Повторный ввод 00 отвергается: элемент уже взят с кратностью $k_U(00) = 2$. Для 11 свободно 3, но остаток 2, поэтому диапазон $[1, 2]$. Итог: $A = \{00^2, 11^2\}$. После каждой ошибки остаток не меняется, и ввод продолжается с того же места.

```

Выберите пункт (0..6): 2

--- Заполнение A ---
Способ заполнения:
  1 - вручную
  2 - автоматически
  0 - отмена
Способ (0..2): 1
Мощность |A| (0..7; Enter - случайная): 4
  №   Код   k_U
    0   00    2
    1   01    1
    2   11    3
    3   10    1
Мощности: |U| = 7
Вводите код элемента и добавляемую кратность; пустая строка вместо кода - отмена.
Осталось распределить: 4
  Код элемента: 0
  Ошибка: код должен состоять из 2 символов 0 и 1
  Код элемента: 0a
  Ошибка: код должен состоять из 2 символов 0 и 1
  Код элемента: 00
  Кратность (1..2): 3
  Ошибка: введите целое число от 1 до 2
  Кратность (1..2): 2
Осталось распределить: 2
  Код элемента: 00
  Ошибка: элемент 00 уже входит в A с наибольшей кратностью k_U(00) = 2
Осталось распределить: 2
  Код элемента: 11
  Кратность (1..2): 2
A = {00(2), 11(2)}, |A| = 4

```

Рисунок 8 — Ручное заполнение A с ошибками ввода

Для B выбирается ручное заполнение со случайной мощностью ($|B| = 3$), но на запрос кода вводится пустая строка: заполнение отменяется, и в меню B по-прежнему «не заполнено». Затем B заполняется автоматически с мощностью 3: $B = \{00^2, 11^1\}$ (рисунок 9).

```

Выберите пункт (0..6): 3

--- Заполнение B ---
Способ заполнения:
  1 - вручную
  2 - автоматически
  0 - отмена
Способ (0..2): 1
Мощность |B| (0..7; Enter - случайная):
Мощность выбрана случайно: |B| = 3
  №   Код   k_U
  0    00    2
  1    01    1
  2    11    3
  3    10    1
Мощности: |U| = 7
Вводите код элемента и добавляемую кратность; пустая строка вместо кода - отмена.
Осталось распределить: 3
  Код элемента:
Заполнение отменено, B не изменено.

=====
          Мультимножества на бинарном коде Грея
=====
U: n = 2, элементов 4, |U| = 7
A: |A| = 4
B: не заполнено
-----
1. Создать универсум U
2. Заполнить A
3. Заполнить B
4. Таблица кратностей U, A, B
5. Операции над A и B
6. Таблица кратностей результатов операций
0. Выход
-----
Выберите пункт (0..6): 3

--- Заполнение B ---
Способ заполнения:
  1 - вручную
  2 - автоматически
  0 - отмена
Способ (0..2): 2
Мощность |B| (0..7; Enter - случайная): 3
B = {00(2), 11(1)}, |B| = 3

```

Рисунок 9 — Отмена ручного заполнения B и автоматическое заполнение

Результаты — на рисунках 10 и 11. Для 00 ($k_U = k_A = k_B = 2$): $k_{A \setminus B} = \min\{2, 0\} = 0$, хотя элемент есть в A , — он целиком «занят» в B . Для 11 ($k_U = 3, k_A = 2, k_B = 1$): $k_{A \setminus B} = \min\{2, 2\} = 2, k_{A-B} = 1, k_{A+B} = \min\{3, 3\} = 3, k_{A/B} = 2, k_{B/A} = 0$.

Выберите пункт (0..6): 5

--- Операции над A и B ---

$U = \{00(2), 01(1), 11(3), 10(1)\}, |U| = 7$
 $A = \{00(2), 11(2)\}, |A| = 4$
 $B = \{00(2), 11(1)\}, |B| = 3$

Теоретико-множественные операции:

Объединение	$A \cup B$	$= \{00(2), 11(2)\},$ мощность 4
Пересечение	$A \cap B$	$= \{00(2), 11(1)\},$ мощность 3
Разность	$A \setminus B$	$= \{11(2)\},$ мощность 2
Разность	$B \setminus A$	$= \{11(1)\},$ мощность 1
Симметрическая разность	$A \Delta B$	$= \{11(2)\},$ мощность 2
Дополнение	$\neg A$	$= \{01(1), 11(1), 10(1)\},$ мощность 3
Дополнение	$\neg B$	$= \{01(1), 11(2), 10(1)\},$ мощность 4

Арифметические операции:

Арифметическая сумма	$A + B$	$= \{00(2), 11(3)\},$ мощность 5
Арифметическая разность	$A - B$	$= \{11(1)\},$ мощность 1
Арифметическая разность	$B - A$	$= \{\},$ мощность 0
Арифметическое произведение	$A * B$	$= \{00(2), 11(2)\},$ мощность 4
Арифметическое деление	A / B	$= \{00(1), 11(2)\},$ мощность 3
Арифметическое деление	B / A	$= \{00(1)\},$ мощность 1

Рисунок 10 — Результаты операций (сценарий 2)

Выберите пункт (0..6): 6

--- Кратности элементов в результатах операций ---

Код	U	A	B	$A \cup B$	$A \cap B$	$A \setminus B$	$B \setminus A$	$A \Delta B$	$\neg A$	$\neg B$	$A+B$	$A-B$	$B-A$	$A*B$	A/B	B/A
00	2	2	2	2	2	0	0	0	0	0	2	0	0	2	1	1
01	1	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0
11	3	2	1	2	1	2	1	2	1	2	3	1	0	2	2	0
10	1	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0
$ \cdot $	7	4	3	4	3	2	1	2	3	4	5	1	0	4	3	1

Рисунок 11 — Таблица кратностей результатов (сценарий 2)

3.3 Сценарий 3: универсум по мощности, граничные мощности A и B

Универсум с $n = 3$ задаётся по мощности (способ 3, рисунок 12). Значения 7 и 801 вне отрезка $[2^3, 100 \cdot 2^3] = [8, 800]$ отвергаются, принимается $|U| = 20$; кратности распределены случайно, каждая не меньше 1, сумма равна 20.

```

Выберите пункт (0..6): 1

--- Создание универсума ---
Разрядность кода Грея n (0..19): 3
Код Грея построен, число элементов 2^n = 8.
Способ задания кратностей k_U(x):
  1 - случайно, от 1 до K
  2 - вручную, для каждого элемента
  3 - по мощности |U|, кратности распределяются случайно
  0 - отмена
Способ (0..3): 3
Мощность |U| (8..800; Enter - случайная): 7
  Ошибка: введите целое число от 8 до 800
Мощность |U| (8..800; Enter - случайная): 801
  Ошибка: введите целое число от 8 до 800
Мощность |U| (8..800; Enter - случайная): 20
Универсум создан.

```

№	Код	k_U
0	000	1
1	001	1
2	011	1
3	010	3
4	110	5
5	111	2
6	101	3
7	100	4

```

Мощности: |U| = 20

```

Рисунок 12 — Универсум заданной мощности

Мощность $|A| = 21 > |U|$ отвергается; при $|A| = 20 = |U|$ получается $A = U$.
 Для B задана мощность 0: $B = \{\}$ (рисунок 13).

```

Выберите пункт (0..6): 2

--- Заполнение A ---
Способ заполнения:
  1 - вручную
  2 - автоматически
  0 - отмена
Способ (0..2): 2
Мощность |A| (0..20; Enter - случайная): 21
  Ошибка: введите целое число от 0 до 20
Мощность |A| (0..20; Enter - случайная): 20
A = {000(1), 001(1), 011(1), 010(3), 110(5), 111(2), 101(3), 100(4)}, |A| = 20

=====
      Мультимножества на бинарном коде Грея
=====
U: n = 3, элементов 8, |U| = 20
A: |A| = 20
B: не заполнено

-----
  1. Создать универсум U
  2. Заполнить A
  3. Заполнить B
  4. Таблица кратностей U, A, B
  5. Операции над A и B
  6. Таблица кратностей результатов операций
  0. Выход
-----

Выберите пункт (0..6): 3

--- Заполнение B ---
Способ заполнения:
  1 - вручную
  2 - автоматически
  0 - отмена
Способ (0..2): 2
Мощность |B| (0..20; Enter - случайная): 0
B = {}, |B| = 0

```

Рисунок 13 — Граничные мощности: $A = U, B = \emptyset$

Результаты (рисунки 14, 15) совпадают с ожидаемыми для $A = U, B = \emptyset$:
 $A \cup B = A \setminus B = A \triangle B = \overline{B} = A + B = A - B = U$; $A \cap B = B \setminus A = \overline{A} = B - A = A * B = \emptyset$. Оба частных пусты: A/B — так как все $k_B = 0$, B/A — так как $\lfloor 0/k \rfloor = 0$.

```

Выберите пункт (0..6): 5

--- Операции над A и B ---
U = {000(1), 001(1), 011(1), 010(3), 110(5), 111(2), 101(3), 100(4)}, |U| = 20
A = {000(1), 001(1), 011(1), 010(3), 110(5), 111(2), 101(3), 100(4)}, |A| = 20
B = {}, |B| = 0

Теоретико-множественные операции:
Объединение      A ∪ B = {000(1), 001(1), 011(1), 010(3), 110(5), 111(2), 101(3), 100(4)}, мощность 20
Пересечение       A ∩ B = {}, мощность 0
Разность          A \ B = {000(1), 001(1), 011(1), 010(3), 110(5), 111(2), 101(3), 100(4)}, мощность 20
Разность          B \ A = {}, мощность 0
Симметрическая разность A Δ B = {000(1), 001(1), 011(1), 010(3), 110(5), 111(2), 101(3), 100(4)}, мощность 20
Дополнение        ¬A = {}, мощность 0
Дополнение        ¬B = {000(1), 001(1), 011(1), 010(3), 110(5), 111(2), 101(3), 100(4)}, мощность 20

Арифметические операции:
Арифметическая сумма      A + B = {000(1), 001(1), 011(1), 010(3), 110(5), 111(2), 101(3), 100(4)}, мощность 20
Арифметическая разность    A - B = {000(1), 001(1), 011(1), 010(3), 110(5), 111(2), 101(3), 100(4)}, мощность 20
Арифметическая разность    B - A = {}, мощность 0
Арифметическое произведение A * B = {}, мощность 0
Арифметическое деление     A / B = {}, мощность 0
Арифметическое деление     B / A = {}, мощность 0

```

Рисунок 14 — Результаты операций при $A = U, B = \emptyset$

Выберите пункт (0..6): 6

--- Кратности элементов в результатах операций ---

Код	U	A	B	A∪B	A∩B	A\B	B\A	AΔB	¬A	¬B	A+B	A-B	B-A	A*B	A/B	B/A
000	1	1	0	1	0	1	0	1	0	1	1	1	0	0	0	0
001	1	1	0	1	0	1	0	1	0	1	1	1	0	0	0	0
011	1	1	0	1	0	1	0	1	0	1	1	1	0	0	0	0
010	3	3	0	3	0	3	0	3	0	3	3	3	0	0	0	0
110	5	5	0	5	0	5	0	5	0	5	5	5	0	0	0	0
111	2	2	0	2	0	2	0	2	0	2	2	2	0	0	0	0
101	3	3	0	3	0	3	0	3	0	3	3	3	0	0	0	0
100	4	4	0	4	0	4	0	4	0	4	4	4	0	0	0	0
·	20	20	0	20	0	20	0	20	0	20	20	20	0	0	0	0

Рисунок 15 — Таблица кратностей при $A = U, B = \emptyset$

3.4 Сценарий 4: некорректный ввод в меню и при создании универсума

До создания универсума выбираются пункты 2 и 5; программа выводит подсказку и возвращается в меню (рисунок 16).

```

Выберите пункт (0..6): 2

Сначала создайте универсум (пункт 1).

=====
Мультимножества на бинарном коде Грея
=====
U: не создан
A: не заполнено
B: не заполнено
-----
1. Создать универсум U
2. Заполнить A
3. Заполнить B
4. Таблица кратностей U, A, B
5. Операции над A и B
6. Таблица кратностей результатов операций
0. Выход
-----
Выберите пункт (0..6): 5

Сначала создайте универсум (пункт 1).

```

Рисунок 16 — Пункты 2 и 5 до создания универсума

Как номер пункта вводятся «abc», 7, «-1», пустая строка, «1.5» и «2 3»; каждая строка отвергается сообщением «Ошибка: введите целое число от 0 до 6» (рисунок 17).

```

Выберите пункт (0..6): abc
Ошибка: введите целое число от 0 до 6
Выберите пункт (0..6): 7
Ошибка: введите целое число от 0 до 6
Выберите пункт (0..6): -1
Ошибка: введите целое число от 0 до 6
Выберите пункт (0..6):
Ошибка: введите целое число от 0 до 6
Выберите пункт (0..6): 1.5
Ошибка: введите целое число от 0 до 6
Выберите пункт (0..6): 2 3
Ошибка: введите целое число от 0 до 6
Выберите пункт (0..6): 1

```

Рисунок 17 — Некорректный ввод номера пункта

Как разрядность вводятся 20 (больше $n_{\max} = 19$), «-2» и «abc»; как способ — 4; как K — 0 и 101. Всё отвергается с указанием допустимого диапазона (рисунок 18). При $n = 3$, $K = 1$ все кратности равны 1: универсум — обычное множество, $|U| = 8$.

```

--- Создание универсума ---
Разрядность кода Грея n (0..19): 20
  Ошибка: введите целое число от 0 до 19
Разрядность кода Грея n (0..19): -2
  Ошибка: введите целое число от 0 до 19
Разрядность кода Грея n (0..19): abc
  Ошибка: введите целое число от 0 до 19
Разрядность кода Грея n (0..19): 3
Код Грея построен, число элементов  $2^n = 8$ .
Способ задания кратностей  $k_U(x)$ :
  1 - случайно, от 1 до K
  2 - вручную, для каждого элемента
  3 - по мощности  $|U|$ , кратности распределяются случайно
  0 - отмена
Способ (0..3): 4
  Ошибка: введите целое число от 0 до 3
Способ (0..3): 1
Наибольшая кратность K (1..100): 0
  Ошибка: введите целое число от 1 до 100
Наибольшая кратность K (1..100): 101
  Ошибка: введите целое число от 1 до 100
Наибольшая кратность K (1..100): 1
Универсум создан.
      №   Код   k_U
      0   000    1
      1   001    1
      2   011    1
      3   010    1
      4   110    1
      5   111    1
      6   101    1
      7   100    1
Мощности:  $|U| = 8$ 

```

Рисунок 18 — Некорректный ввод разрядности, способа и K ; универсум при $K = 1$

Пункт 5 при незаполненных A и B выводит подсказку «Сначала заполните A и B (пункты 2 и 3).»; при заполнении A способ 3 отвергается, а 0 отменяет заполнение (рисунок 19).

```

Выберите пункт (0..6): 5

Сначала заполните A и B (пункты 2 и 3).

=====
      Мультимножества на бинарном коде Грея
=====
U: n = 3, элементов 8, |U| = 8
A: не заполнено
B: не заполнено
-----
1. Создать универсум U
2. Заполнить A
3. Заполнить B
4. Таблица кратностей U, A, B
5. Операции над A и B
6. Таблица кратностей результатов операций
0. Выход
-----
Выберите пункт (0..6): 2

--- Заполнение A ---
Способ заполнения:
  1 - вручную
  2 - автоматически
  0 - отмена
Способ (0..2): 3
      Ошибка: введите целое число от 0 до 2
Способ (0..2): 0
Заполнение отменено, A не изменено.

```

Рисунок 19 — Пункт 5 до заполнения и отмена заполнения A

3.5 Сценарий 5: разрядность $n = 0$

При $n = 0$ программа сообщает, что код Грея не содержит ни одного слова, выводит $U = \{\}$, $|U| = 0$ и не запрашивает способ задания кратностей; таблица не содержит строк (рисунок 20). В меню универсум отмечен как созданный: « $n = 0$, элементов 0, $|U| = 0$ ($U = \{\}$)».

```

Выберите пункт (0..6): 1

--- Создание универсума ---
Разрядность кода Грея n (0..19): 0
При n = 0 код Грея не содержит ни одного слова: U = {}, |U| = 0.
Универсум создан.
      №   Код   k_U
(строк нет: при n = 0 носитель пуст)
Мощности: |U| = 0

```

Рисунок 20 — Создание универсума при $n = 0$

A заполняется вручную: мощность 1 отвергается (допустимый отрезок $[0, 0]$), при мощности 0 таблица пуста, цикл ручного ввода не выполняется ни разу,

$A = \{\}$. B заполняется автоматически со случайной мощностью; единственное возможное значение — 0, $B = \{\}$ (рисунок 21).

```

Выберите пункт (0..6): 2

--- Заполнение A ---
Способ заполнения:
  1 - вручную
  2 - автоматически
  0 - отмена
Способ (0..2): 1
Мощность |A| (0..0; Enter - случайная): 1
  Ошибка: введите целое число от 0 до 0
Мощность |A| (0..0; Enter - случайная): 0
  №    Код    k_U
(строк нет: при n = 0 носитель пуст)
Мощности: |U| = 0
Вводите код элемента и добавляемую кратность; пустая строка вместо кода - отмена.
A = {}, |A| = 0

=====
      Мультимножества на бинарном коде Грея
=====
U: n = 0, элементов 0, |U| = 0 (U = {})
A: |A| = 0
B: не заполнено
-----
1. Создать универсум U
2. Заполнить A
3. Заполнить B
4. Таблица кратностей U, A, B
5. Операции над A и B
6. Таблица кратностей результатов операций
0. Выход
-----
Выберите пункт (0..6): 3

--- Заполнение B ---
Способ заполнения:
  1 - вручную
  2 - автоматически
  0 - отмена
Способ (0..2): 2
Мощность |B| (0..0; Enter - случайная):
Мощность выбрана случайно: |B| = 0
B = {}, |B| = 0

```

Рисунок 21 — Ручное и автоматическое заполнение при $n = 0$

Таблица пункта 4 содержит только строку мощностей $|U| = |A| = |B| = 0$ (рисунок 22). Все тринадцать результатов равны $\{\}$ с мощностью 0, таблица результатов содержит только нулевую строку мощностей (рисунок 23), что совпадает с подразделом 1.4.

```

Выберите пункт (0..6): 4

№    Код    k_U    k_A    k_B
(строк нет: при n = 0 носитель пуст)
Мощности: |U| = 0, |A| = 0, |B| = 0

```

Рисунок 22 — Таблица кратностей при $n = 0$

```

Выберите пункт (0..6): 5

--- Операции над A и B ---
U = {}, |U| = 0
A = {}, |A| = 0
B = {}, |B| = 0

Теоретико-множественные операции:
Объединение      A ∪ B = {}, мощность 0
Пересечение       A ∩ B = {}, мощность 0
Разность          A \ B = {}, мощность 0
Разность          B \ A = {}, мощность 0
Симметрическая разность A Δ B = {}, мощность 0
Дополнение        ¬A = {}, мощность 0
Дополнение        ¬B = {}, мощность 0

Арифметические операции:
Арифметическая сумма      A + B = {}, мощность 0
Арифметическая разность   A - B = {}, мощность 0
Арифметическая разность   B - A = {}, мощность 0
Арифметическое произведение A * B = {}, мощность 0
Арифметическое деление    A / B = {}, мощность 0
Арифметическое деление    B / A = {}, мощность 0

=====
Мультимножества на бинарном коде Грея
=====
U: n = 0, элементов 0, |U| = 0 (U = {})
A: |A| = 0
B: |B| = 0

-----
1. Создать универсум U
2. Заполнить A
3. Заполнить B
4. Таблица кратностей U, A, B
5. Операции над A и B
6. Таблица кратностей результатов операций
0. Выход
-----

Выберите пункт (0..6): 6

--- Кратности элементов в результатах операций ---
Код    U    A    B    A∪B  A∩B  A\B  B\A  AΔB  ¬A  ¬B  A+B  A-B  B-A  A*B  A/B  B/A
(строк нет: при n = 0 носитель пуст)
|·|    0    0    0    0    0    0    0    0    0    0    0    0    0    0    0    0

```

Рисунок 23 — Результаты всех операций и таблица результатов при $n = 0$

3.6 Сценарий 6: пересоздание универсума и конец ввода

Над универсумом $n = 2, K = 3$ ($|U| = 8$) заполняются $A = \{00^2, 01^2\}$ и $B = \{00^1, 01^1, 10^1\}$ (рисунок 24). Затем создаётся универсум $n = 3, K = 2$: программа сообщает, что A и B были построены над прежним универсумом

и очищены, и выводит новую таблицу из восьми элементов, $|U| = 12$ (рисунок 25). Мультимножества старого носителя не могут попасть в операции с новым универсумом: их признаки сняты, а при попытке смешать носители ядро выбросило бы `CarrierMismatchError`.

```

Выберите пункт (0..6): 2

--- Заполнение A ---
Способ заполнения:
  1 - вручную
  2 - автоматически
  0 - отмена
Способ (0..2): 2
Мощность |A| (0..8; Enter - случайная): 4
A = {00(2), 01(2)}, |A| = 4

=====
                Мультимножества на бинарном коде Грея
=====
U: n = 2, элементов 4, |U| = 8
A: |A| = 4
B: не заполнено

-----
1. Создать универсум U
2. Заполнить A
3. Заполнить B
4. Таблица кратностей U, A, B
5. Операции над A и B
6. Таблица кратностей результатов операций
0. Выход

-----
Выберите пункт (0..6): 3

--- Заполнение B ---
Способ заполнения:
  1 - вручную
  2 - автоматически
  0 - отмена
Способ (0..2): 2
Мощность |B| (0..8; Enter - случайная): 3
B = {00(1), 01(1), 10(1)}, |B| = 3

```

Рисунок 24 — Заполнение A и B над универсумом с $n = 2$

```

Выберите пункт (0..6): 1

--- Создание универсума ---
Разрядность кода Грея n (0..19): 3
Код Грея построен, число элементов 2^n = 8.
Способ задания кратностей k_U(x):
  1 - случайно, от 1 до K
  2 - вручную, для каждого элемента
  3 - по мощности |U|, кратности распределяются случайно
  0 - отмена
Способ (0..3): 1
Наибольшая кратность K (1..100): 2
Универсум создан.
A и B были построены над прежним универсумом и очищены.

```

№	Код	k_U
0	000	1
1	001	1
2	011	2
3	010	2
4	110	1
5	111	2
6	101	2
7	100	1

```

Мощности: |U| = 12

```

Рисунок 25 — Пересоздание универсума с $n = 3$

Начинается ручное заполнение A мощности 2, и на запросе кода поток ввода заканчивается (в консоли — Ctrl+Z и Enter, при вводе из файла — конец файла). Программа выводит «Ввод завершён.» и завершается с кодом 0, не заикливаясь на повторных запросах (рисунок 26).

```

Выберите пункт (0..6): 2

--- Заполнение A ---
Способ заполнения:
  1 - вручную
  2 - автоматически
  0 - отмена
Способ (0..2): 1
Мощность |A| (0..12; Enter - случайная): 2

```

№	Код	k_U
0	000	1
1	001	1
2	011	2
3	010	2
4	110	1
5	111	2
6	101	2
7	100	1

```

Мощности: |U| = 12
Вводите код элемента и добавляемую кратность; пустая строка вместо кода - отмена.
Осталось распределить: 2
Код элемента:
Ввод завершён.

```

Рисунок 26 — Конец ввода во время ручного заполнения

3.7 Сценарий 7: наибольшая разрядность $n = 19$

Создаётся универсум $n = 19$, $K = 100$: $2^{19} = 524\,288$ элементов, $|U| = 26\,489\,557$. Таблица обрезана после 32 строк с пометкой (рисунок 27). A заполняется автоматически со случайной мощностью $|A| = 21\,290\,598$; запись A обрезана после 16 элементов с указанием числа скрытых (рисунок 28). Весь сценарий, включая заполнение и вывод, выполняется за доли секунды, что согласуется с таблицей 5.

```
Выберите пункт (0..6): 1

--- Создание универсума ---
Разрядность кода Грея n (0..19): 19
Код Грея построен, число элементов 2^n = 524288.
Способ задания кратностей k_U(x):
  1 - случайно, от 1 до K
  2 - вручную, для каждого элемента
  3 - по мощности |U|, кратности распределяются случайно
  0 - отмена
Способ (0..3): 1
Наибольшая кратность K (1..100): 100
Универсум создан.

  №   Код                               k_U
  0   00000000000000000000             8
  1   00000000000000000001            23
  2   00000000000000000011            78
  3   00000000000000000010            32
  4   000000000000000000110           44
  5   000000000000000000111           98
  6   000000000000000000101           73
  7   000000000000000000100           46
  8   0000000000000000001100          98
  9   0000000000000000001101          31
 10   0000000000000000001111          54
 11   0000000000000000001110          27
 12   0000000000000000001010          51
 13   0000000000000000001011           9
 14   0000000000000000001001           8
 15   0000000000000000001000          42
 16   00000000000000000011000         27
 17   00000000000000000011001         2
 18   00000000000000000011011         50
 19   00000000000000000011010         53
 20   00000000000000000011110         68
 21   00000000000000000011111         87
 22   00000000000000000011101         81
 23   00000000000000000011100         34
 24   00000000000000000010100         39
 25   00000000000000000010101         40
 26   00000000000000000010111         7
 27   00000000000000000010110         68
 28   00000000000000000010010         29
 29   00000000000000000010011         68
 30   00000000000000000010001         91
 31   00000000000000000010000         70

... показаны первые 32 строк из 524288
Мощности: |U| = 26489557
```

Рисунок 27 — Универсум при $n = 19$, $K = 100$

```

Выберите пункт (0..6): 2

--- Заполнение A ---
Способ заполнения:
  1 - вручную
  2 - автоматически
  0 - отмена
Способ (0..2): 2
Мощность |A| (0..26489557; Enter - случайная):
Мощность выбрана случайно: |A| = 21290598
A = {000000000000000000(7), 000000000000000001(18), 000000000000000011(58), 000000000000000010(27), 0000000000000000
110(36), 0000000000000000111(81), 0000000000000000101(63), 0000000000000000100(41), 00000000000000001100(82), 0000000000
00001101(24), 00000000000000001111(45), 00000000000000001110(21), 00000000000000001010(41), 00000000000000001011(9), 000000
00000001001(5), 00000000000000001000(31), ... ещё 522969 эл.}, |A| = 21290598

```

Рисунок 28 — Автоматическое заполнение A при $n = 19$

ЗАКЛЮЧЕНИЕ

Разработана консольная программа на C++, которая строит бинарный код Грея разрядности от 0 до 19 по номеру инвертируемого разряда, задаёт на нём универсум-мультимножество одним из трёх способов (случайные кратности от 1 до K , ручной ввод кратностей, заданная или случайная мощность), заполняет A и B вручную или автоматически с заданной или случайной мощностью от 0 до $|U|$ и выводит тринадцать результатов операций, таблицы кратностей и мощности. Работа проверена на семи воспроизводимых сценариях: результаты совпали с ручным расчётом (таблица 12), с ожидаемыми значениями в граничных случаях $A = U$, $B = \emptyset$, $n = 0$, $K = 1$, а все некорректные значения отвергнуты с сообщением и повторным запросом.

Достоинства и недостатки.

- Генерация кода Грея не использует рекурсию и не хранит код меньшего порядка: каждое слово получается из предыдущего инвертированием одного разряда, всего $O(2^n)$ действий. Недостаток: слова выдаются только подряд; получить слово по номеру без предыдущих шагов алгоритм не позволяет (это делает формула $i \oplus \lfloor i/2 \rfloor$).
- Мультимножество хранится функцией кратности по номерам носителя: операции выполняются за $O(2^n)$ без поиска, универсум — объект того же класса, а результат любой операции остаётся подмультимножеством U (утверждение подраздела 1.3). Цена — память $O(2^n)$ на каждое мультимножество, даже почти пустое.
- Поиск элемента по коду выполняется за $O(n)$ через обратную таблицу `positions` ценой ещё 2^n чисел памяти.
- Случайное заполнение даёт точно заданную мощность, все наборы «единиц кратности» равновероятны, холостых выборов нет. Недостаток: время $O(|U|)$, а не $O(2^n)$; именно оно ограничивает $n_{\max} = 19$.
- Ручной ввод универсума требует 2^n чисел и практичен только при малых n ; при больших n вывод обрезается, и весь результат на экране не увидеть.
- При $n = 0$ принято $U = \{\}$, тогда как формально множество слов длины 0 состоит из одного пустого слова; соглашение выбрано потому, что пустое слово нельзя ни показать, ни ввести.

Масштабирование. Функциональность расширяется незначительными изменениями кода, так как математическая часть не зависит от интерфейса, все

операции выражены через кратности по номерам, а меню строит вывод и таблицы по общему списку операций.

- *Новая операция* (умножение мультимножества на число λ : $\min\{\lambda k_A(x), k_U(x)\}$; проверка включения $A \subseteq B$ или равенства $A = B$): один метод `Multiset` с одним циклом по кратностям (для проверок уже есть `is_submultiset_of`) и одна строка в `Menu::compute_operations`; пункты 5 и 6 подхватят её автоматически.
- *Больше двух мультимножеств*: поля `a`, `b` заменяются массивом мультимножеств с именами и признаками; `fill_multiset` уже получает мультимножество, признак и имя параметрами, поэтому меняются только пункты меню и цикл в `compute_operations`.
- *Другой носитель* (обычный двоичный код, код Грея с другим порядком, произвольные слова): меняется только конструктор `CodeSet`; операции работают с номерами элементов и носитель не используют.
- *Вывод в файл*: `print_multiset` уже пишет в произвольный `std::ostream`; достаточно передать поток в `show_table` и `show_operations` вместо `std::cout`.
- *Пакетный режим*: уже работает — `lab1 --seed N < файл` выполняет сценарий и печатает протокол; так получены все рисунки раздела 3.
- *Большие n* : узкое место — `random_submultiset`. Если вместо просмотра каждой единицы выбирать $k_A(x)$ сразу из гипергеометрического распределения, время станет $O(2^n)$; по замеру `tools/bench.cpp` операции и построение универсума способом 1 укладываются в 1 с при всех измеренных n до 22 включительно (при $n = 22$ — 264 и 47 мс). Меняются функция `random_submultiset` и константа `MAX_DEPTH`; при $n > 31$ нужна 64-разрядная сборка (`MAX_TYPE_DEPTH` вычисляется из размера `size_t` автоматически).
- *Большие кратности*: константа `MAX_MULTIPPLICITY`; при этом нужно повторить замер (время автозаполнения пропорционально $K_{\max}$) и сохранить $K_{\max}^2 < 2^{32}$, чтобы не переполнялось произведение кратностей в 32-разрядной сборке.

В ходе работы освоены построение кода Грея по номеру изменяемого разряда с доказательством корректности, представление мультимножества функцией

кратности и вычисление операций над ним, равновероятный выбор подмультимножества заданной мощности алгоритмом выборки, обоснование ограничений программы замером, защищённый построчный ввод и разделение программы на математическую часть и интерфейс.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Новиков, Ф. А. Дискретная математика для программистов : учебник для вузов / Ф. А. Новиков. — 3-е изд. — Санкт-Петербург : Питер, 2009. — 384 с.
- 2 Кнут, Д. Э. Искусство программирования. Т. 2. Получисленные алгоритмы / Д. Э. Кнут. — 3-е изд. — Москва : Вильямс, 2000. — 832 с.
- 3 Nielsen, J. Usability Engineering / J. Nielsen. — Boston : Academic Press, 1993. — 362 p.
- 4 Востров, А. В. Дискретная математика : курс лекций / А. В. Востров ; Санкт-Петербургский политехнический университет Петра Великого. — Санкт-Петербург, 2026. — URL: https://tema.spbstu.ru/dismath_lect/ (дата обращения: 04.10.2026). — Текст : электронный.
- 5 `std::uniform_int_distribution` // cppreference.com : [сайт]. — URL: https://en.cppreference.com/w/cpp/numeric/random/uniform_int_distribution (дата обращения: 04.10.2026). — Текст : электронный.
- 6 `std::getline` // cppreference.com : [сайт]. — URL: https://en.cppreference.com/w/cpp/string/basic_string/getline (дата обращения: 04.10.2026). — Текст : электронный.

ПРИЛОЖЕНИЕ А

Текст программы

Листинг 3 — CMakeLists.txt

```
1 cmake_minimum_required(VERSION 3.10)
2 project(lab1 LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 14)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6
7 set(CORE_SOURCES
8     src/core/GrayCode.cpp
9     src/core/Random.cpp
10    src/core/Multiset.cpp
11    src/core/Universe.cpp
12 )
13
14 add_executable(lab1 src/main.cpp src/ui/Input.cpp src/ui/Menu.cpp
15    ${CORE_SOURCES})
16
17 # замер времени, по которому выбрана MAX_DEPTH
18 add_executable(bench tools/bench.cpp ${CORE_SOURCES})
19 target_include_directories(bench PRIVATE src)
```

Листинг 4 — src/core/Random.hpp

```
1 #pragma once
2 #include <cstdint>
3
4 // зерно для генератора: смесь std::random_device и текущего времени
5 unsigned make_seed();
6
7 // задаёт начальное состояние генератора; одно зерно — одна и та же
  последовательность чисел
8 void seed_random(unsigned seed);
9
10 // равновероятное целое число от 0 до bound - 1; bound должно быть больше
  нуля
11 size_t random_below(size_t bound);
```

Листинг 5 — src/core/Random.cpp

```

1 #include "core/Random.hpp"
2 #include <chrono>
3 #include <random>
4
5
6 // вихрь Мерсенна: период 2^19937 - 1, значения 32-битные (у std::rand всего
  15 бит)
7 static std::mt19937& generator(){
8     static std::mt19937 gen;
9     return gen;
10 }
11
12 unsigned make_seed(){
13     // в MinGW std::random_device детерминирован и при каждом запуске выдаёт
14     // одну и ту же последовательность, поэтому к нему примешивается время
15     std::random_device device;
16     unsigned time = static_cast<unsigned>(
17         std::chrono::high_resolution_clock::now().time_since_epoch().count()
18     );
19     return device() ^ time;
20 }
21 void seed_random(unsigned seed){
22     generator().seed(seed);
23 }
24
25 size_t random_below(size_t bound){
26     std::uniform_int_distribution<size_t> distribution(0, bound - 1);
27     return distribution(generator());
28 }

```

Листинг 6 — src/core/GrayCode.hpp

```

1 #pragma once
2 #include <cstdint>
3 #include <stdexcept>
4 #include <string>
5 #include <vector>
6
7 // наибольшая разрядность, которую предлагает интерфейс; обоснование —
  tools/bench.cpp и отчёт, п. 2.2
8 const size_t MAX_DEPTH = 19;
9
10 // предел типа: 2^depth должно помещаться в size_t
11 const size_t MAX_TYPE_DEPTH = sizeof(size_t) * 8 - 1;
12
13

```

```

14 class GrayCodeError : public std::runtime_error{
15 public:
16     explicit GrayCodeError(const std::string&
17         message):std::runtime_error(message){}
18 };
19
20 class InvalidDepthError : public GrayCodeError{
21 public:
22     InvalidDepthError():GrayCodeError("2^n не помещается в size_t"){ }
23 };
24
25 class CodeIndexError : public GrayCodeError{
26 public:
27     CodeIndexError():GrayCodeError("номер кода вне последовательности"){ }
28 };
29
30 // коды Грея разрядности depth в порядке построения алгоритмом из [Новиков]
31 // (функция Q);
32 // при depth = 0 последовательность пуста
33 std::vector<size_t> generate_gray_code(size_t depth);
34
35 // двоичная запись кода длиной depth, старший разряд слева
36 std::string code_to_string(size_t code, size_t depth);
37
38 // носитель мультимножеств: все 2^n кодов Грея разрядности n
39 class CodeSet{
40     size_t depth;
41     std::vector<size_t> codes;           // codes[i] — i-й код Грея
42     std::vector<size_t> positions;      // positions[c] — номер кода c в
43     // последовательности
44 public:
45     explicit CodeSet(size_t depth);
46
47     size_t get_depth() const;
48     size_t size() const;
49     std::string get_code(size_t i) const;
50     bool find(const std::string& code, size_t& index) const;
51 };

```

Листинг 7 — src/core/GrayCode.cpp

```

1 #include "core/GrayCode.hpp"
2
3

```

```

4 // функция Q из алгоритма: номер разряда (справа, с единицы), который
  // меняется на шаге i,
5 // то есть число нулей в конце двоичной записи i плюс один
6 static size_t changed_bit(size_t i){
7     size_t q = 1;
8     size_t j = i;
9     while(j % 2 == 0){
10         j /= 2;
11         q++;
12     }
13     return q;
14 }
15
16 std::vector<size_t> generate_gray_code(size_t depth){
17     if(depth > MAX_TYPE_DEPTH){
18         throw InvalidDepthError();
19     }
20     std::vector<size_t> codes;
21     // слово длины 0 не содержит разрядов: в программе принято, что при n = 0
    // кодов нет
22     if(depth == 0){
23         return codes;
24     }
25     size_t count = size_t(1) << depth;
26     codes.reserve(count);
27     size_t code = 0; // шкала B = 00...0
28     codes.push_back(code);
29     for(size_t i = 1; i < count; i++){
30         size_t p = changed_bit(i); // номер разряда справа, 1 <=
        p <= depth
31         code ^= size_t(1) << (p - 1); // B[p] := 1 - B[p]
32         codes.push_back(code);
33     }
34     return codes;
35 }
36
37 std::string code_to_string(size_t code, size_t depth){
38     std::string res;
39     for(size_t bit = depth; bit-- > 0;){
40         res += ((code >> bit) & 1) ? '1' : '0';
41     }
42     return res;
43 }
44
45
46 CodeSet::CodeSet(size_t
depth):depth(depth),codes(generate_gray_code(depth)),positions(codes.size()){
47     for(size_t i = 0; i < codes.size(); i++){
48         positions[codes[i]] = i;

```

```

49     }
50 }
51
52 size_t CodeSet::get_depth() const{
53     return depth;
54 }
55
56 size_t CodeSet::size() const{
57     return codes.size();
58 }
59
60 std::string CodeSet::get_code(size_t i) const{
61     if(i >= codes.size()){
62         throw CodeIndexError();
63     }
64     return code_to_string(codes[i], depth);
65 }
66
67 bool CodeSet::find(const std::string& code, size_t& index) const{
68     if(codes.empty() || code.size() != depth){
69         return false;
70     }
71     size_t value = 0;
72     for(size_t i = 0; i < code.size(); i++){
73         if(code[i] != '0' && code[i] != '1') return false;
74         value = value * 2 + (code[i] - '0');
75     }
76     // все 2^n слов длины n встречаются в коде Грея, поэтому номер есть у
    любого value
77     index = positions[value];
78     return true;
79 }

```

Листинг 8 — src/core/Multiset.hpp

```

1 #pragma once
2 #include "core/GrayCode.hpp"
3 #include <cstdint>
4 #include <memory>
5 #include <ostream>
6 #include <stdexcept>
7 #include <string>
8 #include <vector>
9
10
11 class MultisetError : public std::runtime_error{
12 public:

```

```

13     explicit MultisetError(const std::string&
14         message):std::runtime_error(message){}
15 };
16 class MultisetIndexError : public MultisetError{
17 public:
18     MultisetIndexError():MultisetError("номер элемента вне носителя"){ }
19 };
20
21 class CarrierMismatchError : public MultisetError{
22 public:
23     CarrierMismatchError():MultisetError("мультимножества построены над
24         разными носителями"){ }
25 };
26 class NotSubmultisetError : public MultisetError{
27 public:
28     NotSubmultisetError():MultisetError("кратность элемента больше, чем в
29         универсуме"){ }
30 };
31 class CardinalityOverflowError : public MultisetError{
32 public:
33     CardinalityOverflowError():MultisetError("мощность больше мощности
34         ограничивающего мультимножества"){ }
35 };
36
37 // мультимножество над носителем — функция кратности: counts[i] = k(x_i), x_i
38 // — i-й код Грея;
39 // универсум U и мультимножества A, B — объекты этого класса над общим
40 // носителем
41 class Multiset{
42     std::shared_ptr<const CodeSet> carrier;
43     std::vector<size_t> counts;
44
45     void check_same_carrier(const Multiset& other) const;
46     void check_within(const Multiset& universe) const;
47
48 public:
49     explicit Multiset(std::shared_ptr<const CodeSet> carrier);    // все
50     кратности равны нулю
51
52     const CodeSet& get_carrier() const;
53     size_t size() const;                                         // число элементов носителя
54     size_t get_count(size_t i) const;
55     void set_count(size_t i, size_t count);
56     size_t cardinality() const;                                  // сумма кратностей
57     bool is_submultiset_of(const Multiset& other) const;

```

```

55
56     Multiset unite(const Multiset& b) const;
57     Multiset intersect(const Multiset& b) const;
58     Multiset complement(const Multiset& universe) const;
59     Multiset difference(const Multiset& b, const Multiset& universe) const;
60     Multiset symmetric_difference(const Multiset& b, const Multiset&
    universe) const;
61
62     Multiset arithmetic_sum(const Multiset& b, const Multiset& universe)
    const;
63     Multiset arithmetic_difference(const Multiset& b) const;
64     Multiset arithmetic_product(const Multiset& b, const Multiset& universe)
    const;
65     Multiset arithmetic_division(const Multiset& b) const;
66
67     friend Multiset random_submultiset(const Multiset& bound, size_t
    cardinality);
68 };
69
70 // равновероятный выбор cardinality единиц кратности из bound (урновая схема
    без возвращения)
71 Multiset random_submultiset(const Multiset& bound, size_t cardinality);
72
73 // запись вида {000(2), 011(1)}; элементы с нулевой кратностью не выводятся;
74 // после limit элементов вывод обрывается с указанием, сколько элементов не
    показано
75 void print_multiset(std::ostream& out, const Multiset& multiset, size_t
    limit);
76 std::ostream& operator<<(std::ostream& out, const Multiset& multiset);

```

Листинг 9 — src/core/Multiset.cpp

```

1 #include "core/Multiset.hpp"
2 #include "core/Random.hpp"
3 #include <algorithm>
4
5
6 Multiset::Multiset(std::shared_ptr<const CodeSet>
    carrier):carrier(carrier),counts(carrier->size(), 0){}
7
8 void Multiset::check_same_carrier(const Multiset& other) const{
9     if(carrier != other.carrier){
10         throw CarrierMismatchError();
11     }
12 }
13
14 void Multiset::check_within(const Multiset& universe) const{

```

```

15     check_same_carrier(universe);
16     if(!is_submultiset_of(universe)){
17         throw NotSubmultisetError();
18     }
19 }
20
21 const CodeSet& Multiset::get_carrier() const{
22     return *carrier;
23 }
24
25 size_t Multiset::size() const{
26     return counts.size();
27 }
28
29 size_t Multiset::get_count(size_t i) const{
30     if(i >= counts.size()){
31         throw MultisetIndexError();
32     }
33     return counts[i];
34 }
35
36 void Multiset::set_count(size_t i, size_t count){
37     if(i >= counts.size()){
38         throw MultisetIndexError();
39     }
40     counts[i] = count;
41 }
42
43 size_t Multiset::cardinality() const{
44     size_t res = 0;
45     for(size_t i = 0; i < counts.size(); i++){
46         res += counts[i];
47     }
48     return res;
49 }
50
51 bool Multiset::is_submultiset_of(const Multiset& other) const{
52     check_same_carrier(other);
53     for(size_t i = 0; i < counts.size(); i++){
54         if(counts[i] > other.counts[i]) return false;
55     }
56     return true;
57 }
58
59 Multiset Multiset::unite(const Multiset& b) const{
60     check_same_carrier(b);
61     Multiset res(carrier);
62     for(size_t i = 0; i < counts.size(); i++){
63         res.counts[i] = std::max(counts[i], b.counts[i]);

```

```

64     }
65     return res;
66 }
67
68 Multiset Multiset::intersect(const Multiset& b) const{
69     check_same_carrier(b);
70     Multiset res(carrier);
71     for(size_t i = 0; i < counts.size(); i++){
72         res.counts[i] = std::min(counts[i], b.counts[i]);
73     }
74     return res;
75 }
76
77 Multiset Multiset::complement(const Multiset& universe) const{
78     check_within(universe);
79     Multiset res(carrier);
80     for(size_t i = 0; i < counts.size(); i++){
81         res.counts[i] = universe.counts[i] - counts[i];    //  $\geq 0$ , так как  $A \subseteq U$ 
82     }
83     return res;
84 }
85
86 Multiset Multiset::difference(const Multiset& b, const Multiset& universe)
87 const{
88     //  $A \setminus B = A \cap \neg B$ 
89     return intersect(b.complement(universe));
90 }
91
92 Multiset Multiset::symmetric_difference(const Multiset& b, const Multiset&
93 universe) const{
94     //  $A \triangle B = (A \setminus B) \cup (B \setminus A)$ 
95     return difference(b, universe).unite(b.difference(*this, universe));
96 }
97
98 Multiset Multiset::arithmetic_sum(const Multiset& b, const Multiset&
99 universe) const{
100     check_within(universe);
101     b.check_within(universe);
102     Multiset res(carrier);
103     for(size_t i = 0; i < counts.size(); i++){
104         res.counts[i] = std::min(counts[i] + b.counts[i],
105             universe.counts[i]);
106     }
107     return res;
108 }
109
110 Multiset Multiset::arithmetic_difference(const Multiset& b) const{
111     check_same_carrier(b);

```

```

108     Multiset res(carrier);
109     for(size_t i = 0; i < counts.size(); i++){
110         res.counts[i] = (counts[i] > b.counts[i]) ? counts[i] - b.counts[i] :
111             0;
112     }
113     return res;
114 }
115 Multiset Multiset::arithmetic_product(const Multiset& b, const Multiset&
116 universe) const{
117     check_within(universe);
118     b.check_within(universe);
119     Multiset res(carrier);
120     for(size_t i = 0; i < counts.size(); i++){
121         res.counts[i] = std::min(counts[i] * b.counts[i],
122             universe.counts[i]);
123     }
124     return res;
125 }
126 Multiset Multiset::arithmetic_division(const Multiset& b) const{
127     check_same_carrier(b);
128     Multiset res(carrier);
129     for(size_t i = 0; i < counts.size(); i++){
130         res.counts[i] = (b.counts[i] != 0) ? counts[i] / b.counts[i] : 0;
131     }
132     return res;
133 }
134
135 Multiset random_submultiset(const Multiset& bound, size_t cardinality){
136     size_t left = bound.cardinality();    // единиц, которые ещё не
137     просмотрены
138     if(cardinality > left){
139         throw CardinalityOverflowError();
140     }
141     // элемент x_i — это bound[i] одинаковых шаров в урне; единицы
142     просматриваются по очереди,
143     // и каждая берётся с вероятностью need / left (алгоритм S, Кнут, т. 2,
144     п. 3.4.2)
145     Multiset res(bound.carrier);
146     size_t need = cardinality;
147     for(size_t i = 0; i < bound.counts.size() && need > 0; i++){
148         size_t taken = 0;
149         for(size_t unit = 0; unit < bound.counts[i] && need > 0; unit++){
150             if(need == left || random_below(left) < need){
151                 taken++;
152                 need--;
153             }
154         }
155     }

```

```

151         left--;
152     }
153     res.counts[i] = taken;
154 }
155 return res;
156 }
157
158
159 void print_multiset(std::ostream& out, const Multiset& multiset, size_t
limit){
160     const CodeSet& carrier = multiset.get_carrier();
161     size_t shown = 0;
162     size_t hidden = 0;
163     out << "{";
164     for(size_t i = 0; i < multiset.size(); i++){
165         size_t count = multiset.get_count(i);
166         if(count == 0) continue;
167         if(shown == limit){
168             hidden++;
169             continue;
170         }
171         if(shown > 0) out << ", ";
172         out << carrier.get_code(i) << "(" << count << ")";
173         shown++;
174     }
175     if(hidden > 0) out << ", ... ещё " << hidden << " эл.";
176     out << "}";
177 }
178
179 std::ostream& operator<<(std::ostream& out, const Multiset& multiset){
180     print_multiset(out, multiset, multiset.size());
181     return out;
182 }

```

Листинг 10 — src/core/Universe.hpp

```

1 #pragma once
2 #include "core/GrayCode.hpp"
3 #include "core/Multiset.hpp"
4 #include <cstdint>
5 #include <memory>
6
7 // наибольшая кратность элемента универса
8 const size_t MAX_MULTIPLICITY = 100;
9
10
11 class InvalidMultiplicityError : public MultisetError{
12 public:

```

```

13     InvalidMultiplicityError():MultisetError("кратность элемента универсума
14     вне диапазона от 1 до MAX_MULTIPLICITY"){
15 };
16 class UniverseCardinalityError : public MultisetError{
17 public:
18     UniverseCardinalityError():MultisetError("мощность универсума вне
19     диапазона от N до N * MAX_MULTIPLICITY"){
20 };
21
22 // наименьшая и наибольшая мощность универсума над носителем из size
23 // элементов
24 size_t min_universe_cardinality(size_t size);
25 size_t max_universe_cardinality(size_t size);
26
27 // универсум, в котором кратность каждого элемента — случайное число от 1 до
28 // max_multiplicity
29 Multiset random_universe(std::shared_ptr<const CodeSet> carrier, size_t
30 max_multiplicity);
31
32 // универсум заданной мощности: каждый элемент получает кратность 1,
33 // остальные cardinality - N единиц распределяются случайно с ограничением
34 // MAX_MULTIPLICITY
35 Multiset universe_with_cardinality(std::shared_ptr<const CodeSet> carrier,
36 size_t cardinality);

```

Листинг 11 — src/core/Universe.cpp

```

1 #include "core/Universe.hpp"
2 #include "core/Random.hpp"
3
4
5 size_t min_universe_cardinality(size_t size){
6     return size;
7 }
8
9 size_t max_universe_cardinality(size_t size){
10     return size * MAX_MULTIPLICITY;
11 }
12
13 Multiset random_universe(std::shared_ptr<const CodeSet> carrier, size_t
14 max_multiplicity){
15     if(max_multiplicity < 1 || max_multiplicity > MAX_MULTIPLICITY){
16         throw InvalidMultiplicityError();
17     }
18     Multiset universe(carrier);

```

```

18     for(size_t i = 0; i < universe.size(); i++){
19         universe.set_count(i, 1 + random_below(max_multiplicity));
20     }
21     return universe;
22 }
23
24 Multiset universe_with_cardinality(std::shared_ptr<const CodeSet> carrier,
size_t cardinality){
25     size_t size = carrier->size();
26     if(cardinality < min_universe_cardinality(size) || cardinality >
max_universe_cardinality(size)){
27         throw UniverseCardinalityError();
28     }
29     // сверх обязательной единицы у каждого элемента есть MAX_MULTIPLICITY -
1 свободных мест
30     Multiset extra(carrier);
31     for(size_t i = 0; i < size; i++){
32         extra.set_count(i, MAX_MULTIPLICITY - 1);
33     }
34     Multiset universe = random_submultiset(extra, cardinality - size);
35     for(size_t i = 0; i < size; i++){
36         universe.set_count(i, universe.get_count(i) + 1);
37     }
38     return universe;
39 }

```

Листинг 12 — src/ui/Input.hpp

```

1 #pragma once
2 #include <cstdint>
3 #include <stdexcept>
4 #include <string>
5
6 // ввод закончился (Ctrl+Z в Windows, Ctrl+D в Linux, конец файла)
7 class InputClosedError : public std::runtime_error{
8 public:
9     InputClosedError():std::runtime_error("ввод закончился"){ }
10 };
11
12 // переспрашивает, пока не будет введено целое число от min до max
13 size_t read_number(const std::string& prompt, size_t min, size_t max);
14
15 // то же, но пустая строка допустима: тогда возвращается false, value не
меняется
16 bool read_number_or_empty(const std::string& prompt, size_t min, size_t max,
size_t& value);
17

```

```

18 // переспрашивает, пока не будет введён код длины depth из символов 0 и 1;
19 // пустая строка — отказ от ввода, тогда возвращается false
20 bool read_code_or_empty(const std::string& prompt, size_t depth, std::string&
    code);

```

Листинг 13 — src/ui/Input.cpp

```

1 #include "ui/Input.hpp"
2 #include <iostream>
3 #ifdef _WIN32
4 #include <io.h>
5 static bool stdin_is_console(){ return _isatty(0) != 0; }
6 #else
7 #include <unistd.h>
8 static bool stdin_is_console(){ return isatty(0) != 0; }
9 #endif
10
11
12 // печатает приглашение и читает строку без пробелов, табуляций и \r по краям
13 static std::string read_line(const std::string& prompt){
14     std::cout << prompt;
15     std::string line;
16     if(!std::getline(std::cin, line)){
17         throw InputClosedError();
18     }
19     // при вводе из файла строка не отображается консолью, поэтому программа
    выводит её сама
20     static const bool echo = !stdin_is_console();
21     if(echo) std::cout << line.substr(0, line.find('\r')) << std::endl;
22     size_t begin = line.find_first_not_of(" \t\r");
23     if(begin == std::string::npos) return "";
24     size_t end = line.find_last_not_of(" \t\r");
25     return line.substr(begin, end - begin + 1);
26 }
27
28 // строка из 1..9 цифр: число не больше 999 999 999 гарантированно помещается
    в size_t
29 static bool parse_number(const std::string& text, size_t& value){
30     if(text.empty() || text.size() > 9) return false;
31     size_t res = 0;
32     for(size_t i = 0; i < text.size(); i++){
33         if(text[i] < '0' || text[i] > '9') return false;
34         res = res * 10 + (text[i] - '0');
35     }
36     value = res;
37     return true;
38 }

```

```

39
40 // строка длины depth из символов 0 и 1
41 static bool is_code(const std::string& text, size_t depth){
42     if(text.size() != depth) return false;
43     for(size_t i = 0; i < text.size(); i++){
44         if(text[i] != '0' && text[i] != '1') return false;
45     }
46     return true;
47 }
48
49 static void print_range_error(size_t min, size_t max){
50     std::cout << " Ошибка: введите целое число от " << min << " до " << max
51     << std::endl;
52 }
53
54 size_t read_number(const std::string& prompt, size_t min, size_t max){
55     while(true){
56         size_t value = 0;
57         if(parse_number(read_line(prompt), value) && value >= min && value <=
58             max){
59             return value;
60         }
61         print_range_error(min, max);
62     }
63 }
64
65 bool read_number_or_empty(const std::string& prompt, size_t min, size_t max,
66 size_t& value){
67     while(true){
68         std::string line = read_line(prompt);
69         if(line.empty()) return false;
70         size_t res = 0;
71         if(parse_number(line, res) && res >= min && res <= max){
72             value = res;
73             return true;
74         }
75         print_range_error(min, max);
76     }
77 }
78
79 bool read_code_or_empty(const std::string& prompt, size_t depth, std::string&
80 code){
81     while(true){
82         std::string line = read_line(prompt);
83         if(line.empty()) return false;
84         if(is_code(line, depth)){
85             code = line;
86             return true;
87         }
88     }
89 }

```

```

84         std::cout << "  Ошибка: код должен состоять из " << depth << "
           символов 0 и 1" << std::endl;
85     }
86 }

```

Листинг 14 — src/ui/Menu.hpp

```

1  #pragma once
2  #include "core/GrayCode.hpp"
3  #include "core/Multiset.hpp"
4  #include <cstdint>
5  #include <memory>
6  #include <string>
7  #include <vector>
8
9  class Menu{
10     struct Operation{
11         std::string title;           // название операции
12         std::string notation;       // обозначение, например A ∪ B
13         Multiset result;
14     };
15
16     std::shared_ptr<const CodeSet> carrier; // общий носитель U, A и B
17     Multiset universe;
18     Multiset a;
19     Multiset b;
20     bool universe_ready;
21     bool a_ready;
22     bool b_ready;
23
24     void print_menu() const;
25     bool require_universe() const;
26     bool require_multisets() const;
27
28     void create_universe();
29     bool read_universe_manually(Multiset& target) const;
30     void fill_multiset(Multiset& multiset, bool& ready, const std::string&
       name);
31     bool fill_manual(Multiset& target, size_t cardinality, const std::string&
       name) const;
32
33     std::vector<Operation> compute_operations() const;
34     void show_table(bool with_multisets) const;
35     void show_operations() const;
36     void show_operations_table() const;
37
38 public:

```

```

39     Menu();
40     void run();
41 };

```

Листинг 15 — src/ui/Menu.cpp

```

1  #include "ui/Menu.hpp"
2  #include "core/Random.hpp"
3  #include "core/Universe.hpp"
4  #include "ui/Input.hpp"
5  #include <algorithm>
6  #include <iostream>
7  #include <sstream>
8
9
10 static const size_t TABLE_LIMIT = 32;    // наибольшее число строк таблицы на
    экране
11 static const size_t PRINT_LIMIT = 16;    // наибольшее число элементов
    мультимножества в строке
12
13 static const std::string LINE =
    "=====";
14 static const std::string THIN_LINE =
    "-----";
15
16
17 // ширина строки в символах: в UTF-8 продолжения символа имеют вид 10xxxxxx
18 static size_t text_width(const std::string& text){
19     size_t width = 0;
20     for(size_t i = 0; i < text.size(); i++){
21         if((static_cast<unsigned char>(text[i]) & 0xC0) != 0x80) width++;
22     }
23     return width;
24 }
25
26 static std::string pad_right(const std::string& text, size_t width){
27     size_t current = text_width(text);
28     return current >= width ? text : text + std::string(width - current, '
    ');
29 }
30
31 static std::string pad_left(const std::string& text, size_t width){
32     size_t current = text_width(text);
33     return current >= width ? text : std::string(width - current, ' ') +
    text;
34 }
35

```

```

36 static std::string format_multiset(const Multiset& multiset){
37     std::ostringstream out;
38     print_multiset(out, multiset, PRINT_LIMIT);
39     return out.str();
40 }
41
42 static void print_header(const std::string& title){
43     std::cout << "--- " << title << " ---" << std::endl;
44 }
45
46
47 Menu::Menu():carrier(std::make_shared<CodeSet>(0)),universe(carrier),a(carri
er),b(carrier),
48     universe_ready(false),a_ready(false),b_ready(false){}
49
50 void Menu::run(){
51     while(true){
52         print_menu();
53         size_t choice = read_number("Выберите пункт (0..6): ", 0, 6);
54         std::cout << std::endl;
55         switch(choice){
56             case 0:
57                 std::cout << "Работа завершена." << std::endl;
58                 return;
59             case 1:
60                 create_universe();
61                 break;
62             case 2:
63                 if(require_universe()) fill_multiset(a, a_ready, "A");
64                 break;
65             case 3:
66                 if(require_universe()) fill_multiset(b, b_ready, "B");
67                 break;
68             case 4:
69                 if(require_universe()) show_table(true);
70                 break;
71             case 5:
72                 if(require_universe() && require_multisets()) show_operations();
73                 break;
74             case 6:
75                 if(require_universe() && require_multisets())
76                     show_operations_table();
77                 break;
78             }
79         std::cout << std::endl;
80     }
81
82 void Menu::print_menu() const{

```

```

83     std::cout << LINE << std::endl;
84     std::cout << "          Мультимножества на бинарном коде Грея" << std::endl;
85     std::cout << LINE << std::endl;
86     std::cout << "  U: ";
87     if(universe_ready){
88         std::cout << "n = " << carrier->get_depth() << ", элементов " <<
            universe.size()
89             << ", |U| = " << universe.cardinality();
90         if(universe.size() == 0) std::cout << " (U = {})" ;
91         std::cout << std::endl;
92     }
93     else{
94         std::cout << "не создан" << std::endl;
95     }
96     std::cout << "  A: ";
97     if(a_ready) std::cout << "|A| = " << a.cardinality() << std::endl;
98     else std::cout << "не заполнено" << std::endl;
99     std::cout << "  B: ";
100    if(b_ready) std::cout << "|B| = " << b.cardinality() << std::endl;
101    else std::cout << "не заполнено" << std::endl;
102    std::cout << THIN_LINE << std::endl;
103    std::cout << "  1. Создать универсум U" << std::endl;
104    std::cout << "  2. Заполнить A" << std::endl;
105    std::cout << "  3. Заполнить B" << std::endl;
106    std::cout << "  4. Таблица кратностей U, A, B" << std::endl;
107    std::cout << "  5. Операции над A и B" << std::endl;
108    std::cout << "  6. Таблица кратностей результатов операций" << std::endl;
109    std::cout << "  0. Выход" << std::endl;
110    std::cout << THIN_LINE << std::endl;
111 }
112
113 bool Menu::require_universe() const{
114     if(!universe_ready){
115         std::cout << "Сначала создайте универсум (пункт 1)." << std::endl;
116         return false;
117     }
118     return true;
119 }
120
121 bool Menu::require_multisets() const{
122     if(!a_ready || !b_ready){
123         std::cout << "Сначала заполните A и B (пункты 2 и 3)." << std::endl;
124         return false;
125     }
126     return true;
127 }
128
129 void Menu::create_universe(){
130     print_header("Создание универсума");

```

```

131 size_t depth = read_number("Разрядность кода Грея n (0.." +
std::to_string(MAX_DEPTH) + "): ", 0, MAX_DEPTH);
132 std::shared_ptr<const CodeSet> new_carrier =
std::make_shared<CodeSet>(depth);
133 size_t size = new_carrier->size();
134 Multiset new_universe(new_carrier);
135
136 if(size == 0){
137     std::cout << "При n = 0 код Грея не содержит ни одного слова: U = {},
|U| = 0." << std::endl;
138 }
139 else{
140     std::cout << "Код Грея построен, число элементов 2^n = " << size <<
"." << std::endl;
141     std::cout << "Способ задания кратностей k_U(x):" << std::endl;
142     std::cout << " 1 - случайно, от 1 до K" << std::endl;
143     std::cout << " 2 - вручную, для каждого элемента" << std::endl;
144     std::cout << " 3 - по мощности |U|, кратности распределяются
случайно" << std::endl;
145     std::cout << " 0 - отмена" << std::endl;
146     size_t method = read_number("Способ (0..3): ", 0, 3);
147     if(method == 0){
148         std::cout << "Создание отменено, прежнее состояние сохранено." <<
std::endl;
149         return;
150     }
151     if(method == 1){
152         size_t max_multiplicity = read_number("Наибольшая кратность K
(1.." + std::to_string(MAX_MULTIPPLICITY) + "): ",
153                                             1, MAX_MULTIPPLICITY);
154         new_universe = random_universe(new_carrier, max_multiplicity);
155     }
156     else if(method == 2){
157         if(!read_universe_manually(new_universe)){
158             std::cout << "Создание отменено, прежнее состояние
сохранено." << std::endl;
159             return;
160         }
161     }
162     else{
163         size_t low = min_universe_cardinality(size);
164         size_t high = max_universe_cardinality(size);
165         size_t cardinality = 0;
166         if(!read_number_or_empty("Мощность |U| (" + std::to_string(low) +
".." + std::to_string(high)
167                                 + "; Enter - случайная): ", low, high,
cardinality)){
168             cardinality = low + random_below(high - low + 1);

```

```

169         std::cout << "Мощность выбрана случайно: |U| = " <<
            cardinality << std::endl;
170     }
171     new_universe = universe_with_cardinality(new_carrier,
        cardinality);
172 }
173 }
174
175 bool had_multisets = a_ready || b_ready;
176 carrier = new_carrier;
177 universe = new_universe;
178 a = Multiset(carrier);
179 b = Multiset(carrier);
180 universe_ready = true;
181 a_ready = false;
182 b_ready = false;
183
184 std::cout << "Универсум создан." << std::endl;
185 if(had_multisets){
186     std::cout << "А и В были построены над прежним универсумом и
        очищены." << std::endl;
187 }
188 show_table(false);
189 }
190
191 bool Menu::read_universe_manually(Multiset& target) const{
192     std::cout << "Введите кратность каждого элемента (1.." <<
        MAX_MULTIPLICITY
193         << "); пустая строка - отмена." << std::endl;
194     for(size_t i = 0; i < target.size(); i++){
195         size_t count = 0;
196         if(!read_number_or_empty(" k_U(" + target.get_carrier().get_code(i)
            + ") = ", 1, MAX_MULTIPLICITY, count)){
197             return false;
198         }
199         target.set_count(i, count);
200     }
201     return true;
202 }
203
204 void Menu::fill_multiset(Multiset& multiset, bool& ready, const std::string&
    name){
205     print_header("Заполнение " + name);
206     std::cout << "Способ заполнения:" << std::endl;
207     std::cout << " 1 - вручную" << std::endl;
208     std::cout << " 2 - автоматически" << std::endl;
209     std::cout << " 0 - отмена" << std::endl;
210     size_t method = read_number("Способ (0..2): ", 0, 2);
211     if(method == 0){

```

```

212         std::cout << "Заполнение отменено, " << name << " не изменено." <<
        std::endl;
213         return;
214     }
215
216     size_t high = universe.cardinality();
217     size_t cardinality = 0;
218     if(!read_number_or_empty("Мощность |" + name + "|" (0.." +
        std::to_string(high) + "; Enter - случайная): ",
219                             0, high, cardinality)){
220         cardinality = random_below(high + 1);
221         std::cout << "Мощность выбрана случайно: |" << name << "|" = " <<
        cardinality << std::endl;
222     }
223
224     // заполняется копия: при отмене ручного ввода прежнее мультимножество
        сохраняется
225     Multiset filled(carrier);
226     if(method == 1){
227         if(!fill_manual(filled, cardinality, name)){
228             std::cout << "Заполнение отменено, " << name << " не изменено."
                << std::endl;
229             return;
230         }
231     }
232     else{
233         filled = random_submultiset(universe, cardinality);
234     }
235
236     multiset = filled;
237     ready = true;
238     std::cout << name << " = " << format_multiset(multiset) << ", |" << name
        << "|" = "
239             << multiset.cardinality() << std::endl;
240 }
241
242 bool Menu::fill_manual(Multiset& target, size_t cardinality, const
        std::string& name) const{
243     show_table(false);
244     std::cout << "Вводите код элемента и добавляемую кратность; пустая строка
        вместо кода - отмена." << std::endl;
245     size_t depth = carrier->get_depth();
246     while(target.cardinality() < cardinality){
247         size_t left = cardinality - target.cardinality();
248         std::cout << "Осталось распределить: " << left << std::endl;
249
250         std::string code;
251         if(!read_code_or_empty(" Код элемента: ", depth, code)){
252             return false;

```

```

253     }
254     size_t index = 0;
255     carrier->find(code, index);    // код длины n из 0 и 1 всегда есть в
универсуме
256     size_t free = universe.get_count(index) - target.get_count(index);
257     if(free == 0){
258         std::cout << "  Ошибка: элемент " << code << " уже входит в " <<
name << " с наибольшей кратностью k_U("
259             << code << ") = " << universe.get_count(index) <<
std::endl;
260         continue;
261     }
262     size_t limit = std::min(free, left);
263     size_t count = read_number("  Кратность (1.." + std::to_string(limit)
+ "): ", 1, limit);
264     target.set_count(index, target.get_count(index) + count);
265 }
266 return true;
267 }
268
269 std::vector<Menu::Operation> Menu::compute_operations() const{
270     std::vector<Operation> res;
271     res.push_back({"Объединение", "A  $\cup$  B", a.unite(b)});
272     res.push_back({"Пересечение", "A  $\cap$  B", a.intersect(b)});
273     res.push_back({"Разность", "A  $\setminus$  B", a.difference(b, universe)});
274     res.push_back({"Разность", "B  $\setminus$  A", b.difference(a, universe)});
275     res.push_back({"Симметрическая разность", "A  $\Delta$  B",
a.symmetric_difference(b, universe)});
276     res.push_back({"Дополнение", " $\neg$ A", a.complement(universe)});
277     res.push_back({"Дополнение", " $\neg$ B", b.complement(universe)});
278     res.push_back({"Арифметическая сумма", "A + B", a.arithmetic_sum(b,
universe)});
279     res.push_back({"Арифметическая разность", "A - B",
a.arithmetic_difference(b)});
280     res.push_back({"Арифметическая разность", "B - A",
b.arithmetic_difference(a)});
281     res.push_back({"Арифметическое произведение", "A * B",
a.arithmetic_product(b, universe)});
282     res.push_back({"Арифметическое деление", "A / B",
a.arithmetic_division(b)});
283     res.push_back({"Арифметическое деление", "B / A",
b.arithmetic_division(a)});
284     return res;
285 }
286
287 void Menu::show_table(bool with_multisets) const{
288     size_t depth = carrier->get_depth();
289     size_t code_width = std::max<size_t>(depth, 3);

```

```

290     std::cout << pad_left("№", 8) << "    " << pad_right("Код", code_width) <<
pad_left("k_U", 6);
291     if(with_multisets) std::cout << pad_left("k_A", 6) << pad_left("k_B", 6);
292     std::cout << std::endl;
293
294     size_t rows = std::min(universe.size(), TABLE_LIMIT);
295     for(size_t i = 0; i < rows; i++){
296         std::cout << pad_left(std::to_string(i), 8) << "    " <<
pad_right(carrier->get_code(i), code_width)
297         << pad_left(std::to_string(universe.get_count(i)), 6);
298         if(with_multisets){
299             std::cout << pad_left(a_ready ? std::to_string(a.get_count(i)) :
"-", 6)
300             << pad_left(b_ready ? std::to_string(b.get_count(i)) :
"-", 6);
301         }
302         std::cout << std::endl;
303     }
304     if(universe.size() == 0){
305         std::cout << "    (строк нет: при n = 0 носитель пуст)" << std::endl;
306     }
307     else if(rows < universe.size()){
308         std::cout << "    ... показаны первые " << rows << " строк из " <<
universe.size() << std::endl;
309     }
310
311     std::cout << "Мощности: |U| = " << universe.cardinality();
312     if(with_multisets && a_ready) std::cout << ", |A| = " << a.cardinality();
313     if(with_multisets && b_ready) std::cout << ", |B| = " << b.cardinality();
314     std::cout << std::endl;
315 }
316
317 void Menu::show_operations() const{
318     print_header("Операции над A и B");
319     std::cout << "U = " << format_multiset(universe) << ", |U| = " <<
universe.cardinality() << std::endl;
320     std::cout << "A = " << format_multiset(a) << ", |A| = " <<
a.cardinality() << std::endl;
321     std::cout << "B = " << format_multiset(b) << ", |B| = " <<
b.cardinality() << std::endl;
322
323     std::vector<Operation> operations = compute_operations();
324     for(size_t i = 0; i < operations.size(); i++){
325         if(i == 0) std::cout << std::endl << "Теоретико-множественные
операции:" << std::endl;
326         if(i == 7) std::cout << std::endl << "Арифметические операции:" <<
std::endl;
327         const Operation& op = operations[i];

```

```

328         std::cout << " " << pad_right(op.title, 28) <<
        pad_right(op.notation, 6) << " = "
329             << format_multiset(op.result) << ", мощность " <<
        op.result.cardinality() << std::endl;
330     }
331 }
332
333 void Menu::show_operations_table() const{
334     print_header("Кратности элементов в результатах операций");
335     std::vector<Operation> operations = compute_operations();
336     std::vector<std::string> titles = {"U", "A", "B"};
337     std::vector<const Multiset*> columns = {&universe, &a, &b};
338     for(size_t i = 0; i < operations.size(); i++){
339         std::string title = operations[i].notation;
340         title.erase(std::remove(title.begin(), title.end(), ' '),
        title.end()); // A ∪ B -> AUB
341         titles.push_back(title);
342         columns.push_back(&operations[i].result);
343     }
344
345     size_t code_width = std::max<size_t>(carrier->get_depth(), 3);
346     std::cout << pad_right("Код", code_width);
347     for(size_t j = 0; j < titles.size(); j++) std::cout <<
        pad_left(titles[j], 6);
348     std::cout << std::endl;
349
350     size_t rows = std::min(universe.size(), TABLE_LIMIT);
351     for(size_t i = 0; i < rows; i++){
352         std::cout << pad_right(carrier->get_code(i), code_width);
353         for(size_t j = 0; j < columns.size(); j++){
354             std::cout << pad_left(std::to_string(columns[j]->get_count(i)),
        6);
355         }
356         std::cout << std::endl;
357     }
358     if(universe.size() == 0){
359         std::cout << " (строк нет: при n = 0 носитель пуст)" << std::endl;
360     }
361     else if(rows < universe.size()){
362         std::cout << " ... показаны первые " << rows << " строк из " <<
        universe.size() << std::endl;
363     }
364
365     std::cout << pad_right("|·|", code_width);
366     for(size_t j = 0; j < columns.size(); j++){
367         std::cout << pad_left(std::to_string(columns[j]->cardinality()), 6);
368     }
369     std::cout << std::endl;

```

Листинг 16 — src/main.cpp

```

1 #include "core/Random.hpp"
2 #include "ui/Input.hpp"
3 #include "ui/Menu.hpp"
4 #include <cstdlib>
5 #include <cstring>
6 #include <exception>
7 #include <iostream>
8 #ifdef _WIN32
9 #include <windows.h>
10 #endif
11
12
13 int main(int argc, char* argv[]){
14 #ifdef _WIN32
15     // строки программы записаны в UTF-8, а консоль Windows по умолчанию
16     // выводит в кодировке 866
17     SetConsoleOutputCP(CP_UTF8);
18 #endif
19     // lab1 --seed N повторяет запуск с теми же случайными числами
20     unsigned seed = make_seed();
21     if(argc == 3 && std::strcmp(argv[1], "--seed") == 0){
22         seed = static_cast<unsigned>(std::strtoul(argv[2], nullptr, 10));
23     }
24     seed_random(seed);
25     std::cout << "Зерно генератора: " << seed << " (повторить запуск: lab1
26     --seed " << seed << ")" << std::endl
27     << std::endl;
28
29     try{
30         Menu menu;
31         menu.run();
32     }
33     catch(const InputClosedError&){
34         std::cout << std::endl << "Ввод завершён." << std::endl;
35     }
36     catch(const std::exception& error){
37         std::cout << "Внутренняя ошибка: " << error.what() << std::endl;
38         return 1;
39     }
40     return 0;
41 }

```

Листинг 17 — tools/bench.cpp

```

1 // Замер времени самых долгих действий программы при разных разрядностях n.
2 // По нему выбрана MAX_DEPTH: наибольшее n, при котором любое действие
   занимает меньше 1 с.
3 // Сборка: g++ -std=c++14 -O2 -Isrc tools/bench.cpp src/core/*.cpp -o bench
4 #include "core/GrayCode.hpp"
5 #include "core/Multiset.hpp"
6 #include "core/Random.hpp"
7 #include "core/Universe.hpp"
8 #include <algorithm>
9 #include <chrono>
10 #include <iomanip>
11 #include <iostream>
12 #include <memory>
13
14
15 // наименьшее из трёх измерений, мс: так меньше сказываются посторонние
   процессы
16 template<class Action>
17 static double measure(Action action){
18     double best = 1e18;
19     for(int run = 0; run < 3; run++){
20         auto start = std::chrono::steady_clock::now();
21         action();
22         auto finish = std::chrono::steady_clock::now();
23         best = std::min(best, std::chrono::duration<double,
24             std::milli>(finish - start).count());
25     }
26     return best;
27 }
28
29 int main(){
30     seed_random(1);
31     std::cout << " n код Грея U случайно U по мощности заполнение A
32     13 операций наибольшее, мс" << std::endl;
33     for(size_t depth = 10; depth <= MAX_DEPTH + 2; depth++){
34         std::shared_ptr<const CodeSet> carrier;
35         double t_code = measure([&]{ carrier =
36             std::make_shared<CodeSet>(depth); });
37
38         // наихудший случай: все кратности равны MAX_MULTIPLICITY
39         Multiset universe(carrier);
40         double t_random = measure([&]{ universe = random_universe(carrier,
41             MAX_MULTIPLICITY); });
42         size_t low = min_universe_cardinality(carrier->size());
43         size_t high = max_universe_cardinality(carrier->size());
44         double t_by_cardinality = measure([&]{ universe =
45             universe_with_cardinality(carrier, (low + high) / 2); });

```

```

41     for(size_t i = 0; i < universe.size(); i++) universe.set_count(i,
MAX_MULTIPPLICITY);
42
43     // при мощности |U| / 2 алгоритм просматривает почти все |U| единиц
44     Multiset a(carrier);
45     Multiset b(carrier);
46     double t_fill = measure([&]{ a = random_submultiset(universe,
universe.cardinality() / 2); });
47     b = random_submultiset(universe, universe.cardinality() / 3);
48
49     size_t checksum = 0;
50     double t_operations = measure([&]{
51         checksum += a.unite(b).cardinality() +
a.intersect(b).cardinality()
52             + a.difference(b, universe).cardinality() +
b.difference(a, universe).cardinality()
53             + a.symmetric_difference(b, universe).cardinality()
54             + a.complement(universe).cardinality() +
b.complement(universe).cardinality()
55             + a.arithmetic_sum(b, universe).cardinality() +
a.arithmetic_difference(b).cardinality()
56             + b.arithmetic_difference(a).cardinality() +
a.arithmetic_product(b, universe).cardinality()
57             + a.arithmetic_division(b).cardinality() +
b.arithmetic_division(a).cardinality();
58     });
59
60     double worst = std::max({t_code, t_random, t_by_cardinality, t_fill,
t_operations});
61     std::cout << std::fixed << std::setprecision(1) << std::setw(2) <<
depth
62         << std::setw(11) << t_code << std::setw(13) << t_random <<
std::setw(16) << t_by_cardinality
63         << std::setw(15) << t_fill << std::setw(14) << t_operations
<< std::setw(17) << worst
64         << (worst < 1000 ? "" : " > 1 c") << std::endl;
65     if(checksum == 0) std::cout << "";    // не даёт компилятору выбросить
операции
66 }
67 return 0;
68 }

```